The Art of Software Architecture

133 maxims for building systems that refuse to lie

Compiled from the meta-principles of the Awareness Graph — a field book for architects, operators, and the brave soul who inherited the service nobody admits to owning.

> *The victorious architect does not begin with code. The victorious architect begins by deciding what may be called true.*

Preface — Before the First Deployment

The old books of strategy open with ground, supply, command, deception, and the cost of a prolonged campaign. Software differs chiefly in one comic detail: its generals are often surprised to learn that the battlefield has been running in production for six years.

This is not a book about drawing diagrams. It is a book about the *government of truth through time* — how a system keeps knowing what it is while release after release washes over it. Its enemy is not complexity; complexity can be measured and paid down. The more dangerous enemies are the ones that look like friends: ambiguity that looks convenient, a fallback that looks helpful, a partial write that looks finished, cached state that looks authoritative, and a green status that looks like comfort.

Every maxim here was born from pain. Somewhere, a timeout was mistaken for a failure; a database row was mistaken for ownership; a retry became a siege engine aimed inward; a recovery path depended on the very service it was meant to recover; a button announced permission before permission had been proved. The maxims are what remained after the incident review, once the shouting stopped and the shape of the mistake stood plain.

Read it as a campaign manual. The passages are short because the lesson usually arrives mid-incident, when nobody has appetite for a twelve-page architecture decision record. The humour is deliberate: a trap that makes an architect smile is easier to remember than one embalmed in committee language. And the aim is modest. It is not to prevent every failure — that is a fantasy sold by people who have not yet met distributed time. The aim is to make failures *bounded, visible, classifiable, recoverable, and unable to impersonate success.*

Each maxim is given in three parts: **the law** itself; **the trap**, which is the reasonable-sounding instinct that leads you to break it; and **the discipline**, which is the smallest concrete rule that keeps you honest. The traps are quoted almost as engineers actually say them — because you will recognise your own voice in more than one, and that recognition is the whole point.

Contents

This is a reference, not a novel — it is meant to be kept and consulted, not read at a sitting. Every maxim is numbered and anchored: the index at the back and the `See also` *cross-references link straight to the relevant one. Each maxim carries a severity tag, though you should not expect it to narrow things much — 80 of the 133 are marked critical. That is the quiet lesson of the whole book: in a system that must tell the truth about itself, most mistakes do not merely slow it down. They make it lie.*

I. On Authority and the Ownership of Truth

A system has many hands, but truth must have one owner. The architect who confuses access with authority convenes a court in which every clerk may rewrite the law. Such systems look flexible — right up to the first repair, when every actor is a king and the database is the battlefield they fight on.

1. Storage is not semantic authority — truth belongs to the owning actor, not the backing store

CRITICAL SEVERITY.

The trap. Having etcd access feels like having authority — but access is not ownership. Its familiar disguise: *"I can see the data so I own it"*. A shared datasource (etcd, ScyllaDB, MinIO, local files) is only a source of durable record. It is not automatically the source of truth. Truth belongs to the actor that owns the semantic meaning of the state.

The discipline. Every code path that reads or writes shared state must be traceable to the owning actor's typed API. Direct storage access by non-owners is an architectural violation. Temporary exceptions require an explicit allowlist entry with a named migration target. The allowlist must shrink over time.

2. Identity computation must be invariant — one field, one meaning, one canonical computation, everywhere

CRITICAL SEVERITY.

The trap. Trusting the type without checking the computation — checksum at publish and checksum at verify can be different computations behind the same name. Its familiar disguise: *"Same field name means same thing"*. Identity-bearing fields (checksum, build_id, version, installed_hash, desired_hash, timestamp anchor) must have exactly one semantic meaning and one canonical computation. The computation must be the same at every point where the field is produced, compared, or verified.

The discipline. Identity fields must be produced by exactly one computation. Any code that compares, verifies, or acts on an identity field must use the same canonical computation that produced it. If a transformation occurs between production and verification (packaging, relocation, wrapping), the identity must be recomputed after transformation by the owning actor — never by the verifier independently.

3. Distributed actors doing the same job must converge, not compete — two writers with different state will fight until one wins by accident

CRITICAL SEVERITY.

The trap. Redundancy feels safe — but uncoordinated redundancy is a fight where timing decides the winner, not correctness. Its familiar disguise: *"Everyone should reconcile independently"*. When multiple actors can write the same state, and they have different views of truth, the last writer wins — and the last writer is determined by timing, not correctness. This is different from "who owns the truth" (meta.storage_is_not_semantic_authority) — here both actors believe they are the owner.

The discipline. Reconcilers that write shared state must be leader-gated. When multiple records can exist for the same entity (e.g. INFRASTRUCTURE and SERVICE kind for the same package), there must be a deterministic priority order. Non-leader instances must return early without touching state.

4. Meaning lives in structure — a value projected down to its primitive shape carries the lie of universality

CRITICAL SEVERITY.

The trap. A primitive type feels universal — but the scope/subject/source the value lived inside is part of what it meant; stripping it leaves a shape the consumer reads as global truth. Its familiar disguise: *"Just give me the value"*. Every value lives inside a structure that gives it meaning: a subject (identity-of-WHAT), a source (which canonical registry it came from), a generation marker. The bare primitive — bool, string, sha256, int — is a projection that preserves the shape while discarding the structure. Consumers downstream read the shape as universal: an entry-point checksum becomes a bundle checksum, an inlined kind-list becomes the catalog, a hardcoded list of platforms becomes the platform truth.

The discipline. Every cross-boundary value must preserve the structure that gives it meaning. Subject-aliasing requires typed wrappers or disambiguating field names; catalog-inlining is forbidden in production code when a registry exists. The two flavors have distinct fix shapes but share one root: don't let the shape outlive the structure.

5. Code mirrors of external truth drift silently — derive the set, don't author it

CRITICAL SEVERITY.

The trap. A hardcoded slice is fast, readable, and trivially correct today — so it feels safer than a discovery loop. But the slice is a second authority for the set, and the moment the external source gains a member, the mirror diverges. No compile error, no test failure (the tests use the same mirror), just silent runtime drift that surfaces months later in production. Its familiar disguise: *"A small hand-maintained list of names will stay up to date"*. Whenever a directory, proto enum, etcd prefix, package registry, installed catalog, or external configuration file defines a set of named entities, ANY Go code that enumerates "the canonical members of that set" must derive the enumeration from the source — never hand-maintain a parallel slice/map/switch.

The discipline. Default rule: any code declaring "the canonical set of X" must derive the set from the same authority that owns X. Acceptable authorities are: - filesystem directory scans (os.ReadDir + extension filter) - proto enum value tables (protoreflect or *_name maps) - etcd prefix listings (kv.Get with prefix) - service-published catalogs read at startup or test time

See also: 1, 2, 3

6. The only authoritative correctness check is at the actor that owns the cross-layer intent

CRITICAL SEVERITY.

The trap. Layered verification feels rigorous — node-agent checks the install, controller checks the release, systemd checks the unit. But each layer only verifies its layer's concerns; the cross-layer property (the SERVICE that was supposed to ship is actually running the right binary and answering requests) is owned by no individual layer. Aggregating per-layer green into "system green" is a category error — you've just made the partial true into a partial lie. Its familiar disguise: *"Every layer verified its own concerns, therefore the whole thing is correct"*. The end-to-end argument (Saltzer, Reed & Clark 1984, anticipated by Lampson 1983

§4.1): an authoritative correctness verdict for a cross-layer property MUST come from an actor that owns the cross-layer intent. Intermediate-layer checks are STRICTLY performance optimizations — they catch failures early and reduce load, but they are never proof.

The discipline. For every cross-layer property the system exposes (health, converged, succeeded, available, ready), name the actor that owns that intent and route the verdict through it. No other actor's local verdict may be used as the cross-layer answer.

See also: 1, 25

7. Cached, replicated, or derived state must declare its staleness contract — 'eventually consistent' is a category, not a contract

CRITICAL SEVERITY.

The trap. When you cache or replicate data, the optimistic view is that callers don't really care about being slightly stale — they'll re-read soon, or the staleness window is "small enough." But "small enough" relative to what? Without a named bound, every caller adopts a different assumption — the dashboard refreshes every 5s, the reconciler runs every 30s, the doctor every 5m, each treating the data as authoritative for its own purposes. Each is operating under a different staleness; the system invariant they share becomes undefined. Its familiar disguise: *"It'll catch up eventually"*. Any data source that is NOT the authoritative writer — caches, read replicas, eventually-consistent stores, async projections, derived views, memoized results — has a STALENESS contract. The contract names how far behind the canonical truth the source can be while still being authoritative for a particular kind of decision. Without a named contract, every consumer assumes a different bound; the system's correctness becomes undefined.

The discipline. Every data source that is NOT the authoritative writer must declare three properties:

- STALENESS BOUND — the max age at which the data can still be treated as authoritative. Expressed in time (seconds) or in source events (cycles, watch ticks, replication events).
- SOURCE-OF-TRUTH POINTER — where to refetch when the bound is exceeded or when authoritative-now is required.
- EXCEEDED BEHAVIOR — what to do when the bound is hit but no fresh data is available: refuse to serve, return a "stale" marker, escalate to the source, etc.

See also: 1, 2, 6, 21

8. Physical clocks on different nodes never agree precisely — for causal ordering, use logical clocks; for total order, use consensus

CRITICAL SEVERITY.

The trap. A timestamp feels like an unambiguous moment, so comparing two timestamps feels like determining which event came first. But the two timestamps come from two physical clocks that drift independently, and "first" by clock comparison can disagree with "first" by causality. Code that orders cross-node events by wall-clock timestamps will eventually corrupt — usually silently, occasionally catastrophically, always at the moment NTP corrects. Its familiar disguise: *"Compare wall-clock timestamps to decide which event happened first"*. Lamport's 1978 "Time, Clocks, and the Ordering of Events in a Distributed System" established the result that any system requiring distributed agreement on event ordering CANNOT rely on physical clock comparison. Physical clocks drift, get NTP-corrected, and disagree across nodes by amounts that

routinely exceed the latency between the events being ordered. A timestamp produced on node A and a timestamp produced on node B cannot be reliably compared to determine which event happened first.

The discipline. Code that orders events must classify each event's source:

- Same-node events: wall-clock timestamps from one clock, compared via that clock's own monotonic guarantee. Safe.
- Cross-node events with causal dependency: must use a logical clock or correlation chain. etcd revisions and Raft (term, index) are the recommended primitives.
- Cross-node events without explicit causality: cannot be ordered. Operations that depend on "which came first" must EITHER establish a logical ordering OR be designed to not require the answer (commutative operations, set semantics, last-writer-wins with version vectors).

See also: 2, 3, 28, 29

9. When the authority decision cannot be made, the default is REFUSE — never 'allow because we could not check'

CRITICAL SEVERITY.

The trap. A failed authorization check feels like a system problem, not a security problem — the user did nothing wrong, the RBAC service is just slow, so denying their request feels unfair. But "I could not verify your authority" is the SAFE direction precisely because the alternative is "I let in someone I cannot prove should be in." Default-allow turns every authorization-system failure into a silent escalation; default-deny turns it into a loud availability problem the operator notices and fixes. Its familiar disguise: *"If RBAC times out we let the request through so it does not break"*. Saltzer and Schroeder's 1975 paper "The Protection of Information in Computer Systems" gave eight design principles for secure systems. The second — fail-safe defaults — says that access decisions should be based on permission rather than exclusion, and the default situation should be lack of access. Operationalized: any code path where the authorization decision CANNOT be made must default to REFUSE.

The discipline. Every authorization check must satisfy:

- The check returns one of {ALLOW, DENY, UNCERTAIN}.
- UNCERTAIN is treated identically to DENY by every consumer — there is no third "let it through" branch.
- The UNCERTAIN result is logged at sufficient severity that operators notice repeated occurrences (one failed check is noise; ten per second is an outage signal).
- Bootstrap or break-glass relaxation windows are EXPLICIT, BOUNDED, and AUDIT-LOGGED at the moment of relaxation, not silent defaults.

See also: 7, 48, 51, 81

10. Every actor's privileges must be explicit and minimal — the default for a new actor is NO privileges, not 'whatever was convenient at creation time'

CRITICAL SEVERITY.

The trap. Setting up a new service or AI actor with broad permissions feels efficient — you do not know yet which RPCs it will need, so granting more avoids breaking things. But the act of postponing privilege

restriction is the act of leaving the default at “more than necessary.” The “tighten later” never happens for the same reason scaffolding never gets removed; the broad permission becomes a load-bearing assumption other code grows to depend on. The principle says the SAFE default is zero, and every grant must be earned by an explicit need. Its familiar disguise: *“It is easier to grant cluster admin and tighten later”*. Saltzer and Schroeder’s sixth principle — least privilege — states that every program and every user should operate with the minimum set of privileges necessary to complete the job. Operationalized: the default state for any new actor (service account, AI agent, workflow step, scheduled task) is NO privileges; each privilege must be explicitly granted with a named reason and bounded scope.

The discipline. For every new actor (service account, AI service, workflow, scheduled task, automation hook):

- INITIAL STATE — zero privileges. The creation flow that grants any privilege must require the privilege to be NAMED.
- NAMED REASON — each grant carries a brief reason in the role binding metadata. “Needs to read / globular/services” is a reason; “for the X service” is not.
- BOUNDED SCOPE — privileges name resources, not wildcards, except where the wildcard is structurally necessary (e.g. “list all services” by definition cannot name each).
- PERIODIC REVIEW — privileges granted should have an expiration or review marker; permanent grants are reserved for foundational service accounts.

See also: 1, 9, 23

11. Abstractions that hide engine choices from their callers force callers to either tolerate the hidden choice or bypass the abstraction entirely

CRITICAL SEVERITY.

The trap. A clean abstraction promises the caller does not need to think about the implementation. But “does not need to” and “cannot” are different. Hiding the engine’s choices means the caller CANNOT reason about them; when reality forces a decision that depends on what the engine chose, the caller’s only options are tolerate the hidden choice or bypass the abstraction entirely. The clean interface becomes either the only option (good when truly sufficient) or the wrong option (bad when reality demands more). Its familiar disguise: *“The handler does not need to know it is being retried”*. Eden (Black 1985 §3.4.2) restated Lampson’s “don’t hide power” with a specific gloss. The Eden Programming Language hid the asynchronous IPC layer behind a synchronous procedure-call interface. The async layer was “rarely used” — not because applications did not need asynchrony, but because the interface did not expose it. Asynchrony was structurally invisible to handlers; the few patterns that needed it had to bypass the language and use the kernel directly. The clean interface was either sufficient or a trap.

The discipline. For every abstraction that mediates between an engine layer and a handler layer (workflow engine, interceptor chain, config loader, RBAC interceptor):

- Enumerate the engine-side facts the abstraction computes during its work (attempt number, prior outcomes, retry state, applied middleware list, consulted policy version).
- For each fact, decide whether it should be exposed, withheld, or made opt-in via an explicit query API.
- Default: expose with a documented contract. Withhold only when leaking the fact would create a security or correctness hazard.
- Make the contract visible in the handler-facing API surface, not buried in engine source.

See also: 22, 25, 26, 42

12. All system entities should be named by a uniform scheme — heterogeneous naming creates friction at every API boundary

CRITICAL SEVERITY.

The trap. Different resources are different and feel like they warrant different naming — a node is fundamentally not a file is fundamentally not a service. But every API at every layer that takes "a thing" then has to know which name type it accepts. Every cross-resource operation has to translate. The cost is invisible per-API; it compounds into a system with dozens of overlapping namespaces and a constant tax on every developer who tries to write code that crosses resource boundaries. Its familiar disguise: *"Nodes are UUIDs, services are name+UUID, packages are name+version+build_id, files are paths — and that is fine"*. Eden (Black 1985 §3.2.2) named "all global system entities… uniformly — by capabilities. This includes not only objects, but also Edentypes, nodes (machines), and checksites (disks)." The uniformity was deliberate. The Eden team explicitly considered separate naming for machines and disks and decided against it; the conceptual unity reduced the system's apparent size and made cross-resource operations naturally composable.

The discipline. For new resource types added to the system:

- Default to the existing canonical naming scheme of the closest existing resource. Adding a new naming scheme requires justification.
- If a new scheme is necessary, document the translation primitives between it and the existing schemes; do not let translation be implicit.
- Whenever a new API takes "a thing" as input, prefer the most-uniform name type available (capability handle, opaque ID, fully-qualified resource path) over type-specific shapes.

See also: 1, 2, 25

13. Capability is not intent — an installed binary answers 'can this node run X', never 'should it'

HIGH SEVERITY.

The trap. A present dependency feels like a mandate — but presence is capacity, not a work order. Its familiar disguise: *"The package/binary is here, so start the service"*. A node capability — an installed binary or package, an available GPU, open ports, a cached artifact, a leftover service config — must never be treated as desired placement. Capability answers "can this node run X?"; desired state answers "should this node run X?". Only the second may trigger install, start, keepalive, or restart.

The discipline. No install/start/restart/keepalive action may be triggered by the mere presence of a binary, package, port, or stale local config. Every such action must trace to a controller-owned desired-state record for this node. "yt-dlp exists" permits the media profile; it does not install or start media services. "old service config exists" does not keep a service alive.

See also: 1, 117

14. Membership is admitted, not self-declared — a service or discovered peer never decides its own place in topology

CRITICAL SEVERITY.

The trap. Seeing a thing (yourself, a peer on the LAN) feels like knowing it belongs — observation is not admission. Its familiar disguise: *"I observe myself running / I discovered a peer, therefore it belongs in the cluster"*. A managed service must not decide whether it belongs in the topology from its own local config, package presence, or running process. A discovered peer must not become a member from a scan, a fetched remote /config, or a public-key fetch. Membership comes from controller-owned desired/admitted state.

The discipline. A service must not self-start into the cluster because a package or config exists. A node must not treat a discovered peer as admitted without a signed join plan, a controller admission record, a node-identity binding, and a generation. Discovery may populate candidates; only the owner promotes a candidate to a member. A service must not remove itself on gen=0 empty topology without a bootstrap guard.

See also: 1, 13, 80

15. Each actor repairs only inside its authority lane — observation flows up, decision flows down, projection renders outward

CRITICAL SEVERITY.

The trap. Being able to touch a resource feels like being allowed to repair it — helpfulness across lanes turns the system to soup. Its familiar disguise: *"I can see the problem, so I'll fix it — even though it's another layer's resource"*. A controller, node-agent, doctor, installer, or service-manager may only repair resources inside its declared authority boundary. This is the four-layer model in one line: doctor observes, controller decides, node-agent applies, service-manager supervises local services, installer lays down packages.

The discipline. No implicit cross-layer write. Doctor must not mutate desired state. Installer must not admit topology. Node-agent must not invent desired state. Service-manager must not install a workload the controller did not request. Every mutation must originate from the actor that owns that layer.

See also: 6, 13, 14

16. Commands request transitions — they do not become state

CRITICAL SEVERITY.

The trap. An accepted command feels like the new fact — but a request is not a grant until the engine validates it. Its familiar disguise: *"HTTP says install succeeded, so state = installed"*. A command (Approve, Install, Join, Cancel, Retry, Commit) is only a request to transition. The engine must validate current state, actor authority, expected generation, idempotency key, guards/preconditions, and whether the transition is allowed — before any state changes. InstallSucceeded is accepted only if current state is INSTALLING and operation_id matches the active transaction.

The discipline. No external signal may set state directly. Every command is evaluated against the transition table and its guards; rejection is a first-class outcome.

See also: 14, 51

17. A workflow instance has a single transition writer — only one authority advances it

CRITICAL SEVERITY.

The trap. Letting whoever-has-news write state feels responsive — two writers with different views fight until one wins by accident. Its familiar disguise: *"controller and agent both update workflow state"*. Only one authority may advance a workflow instance at a time, enforced by a lease, compare-and-swap on generation, transaction lock, owner shard, or a durable ownership record. Controller owns the transition; the agent reports observations/callbacks; the controller accepts/rejects and advances.

The discipline. Concurrent transition writers to one instance is a violation. The single writer is established by CAS/lease/ownership.

See also: 3, 38

18. A distributed data substrate's topology is part of its correctness, not a runtime detail — running is not the same as safe

CRITICAL SEVERITY.

The trap. A green process check feels like safety — but for a replicated store, process health says nothing about quorum, replication factor, or durability. Its familiar disguise: *"The service is active/healthy, therefore the data or control plane it backs is safe"*. Distributed data sources are not merely processes to start, stop, or restart. Their membership topology, replication model, quorum rules, placement constraints, and recovery semantics ARE part of correctness. A substrate may be fully "running" while still unsafe: without quorum, under-replicated, split-brained, holding only non-voting members, or one member/disk/zone loss away from losing the data or control plane.

The discipline. Operators and controllers must reason over TOPOLOGY state (quorum, voter/owner counts, replication factor, erasure-set health), not only process/unit health, before claiming a substrate is safe, available, or done converging. Health reporting for a substrate must derive from the substrate's own membership/quorum/replication semantics.

See also: 1, 14, 19, 39

19. High availability is defined by tolerated failure, not by how many members are present

CRITICAL SEVERITY.

The trap. More members feels like more availability — but availability is about what you can LOSE and still serve, not what you can count. Its familiar disguise: *"There are 2 (or N) nodes, so it is HA"*. A topology is HA only if it can lose the claimed number of members, disks, zones, or services while preserving the required read/write/control-plane guarantees. Member count is not the measure; tolerated failure is.

The discipline. HA status must be computed from the substrate's own quorum / replication / erasure semantics and reported together with the fault model it satisfies. A raft store is HA at 3 voters (survives one voter loss), not at 2. An erasure store is durable only to its configured parity. Never emit an HA or "protected" verdict without the tolerated-failure count behind it.

See also: 18, 20, 25, 34

20. Non-voting, non-owning, learner, observer, limited, or partially-initialized members do not count as capacity

CRITICAL SEVERITY.

The trap. A present member feels like available capacity — but a member that cannot vote, own, or store is expansion potential, not guarantee. Its familiar disguise: *"The learner/observer/zero-token/not-yet-healed member exists, so count it toward quorum / RF / durability"*. Members admitted in a limited role — non-voting, non-owning, learner, observer, limited-voter, zero-token, or partially initialized — must NOT count toward HA, quorum, replication factor, storage durability, or founding topology guarantees unless the substrate explicitly says they do.

The discipline. Any computation of "is this HA / RF-safe / durable / quorate" must exclude limited-role members from the eligible set. Founding-quorum and replication-factor eligibility checks must count only full voters/owners.

See also: 14, 19, 75

> *Field note — Granting database credentials to a service does not crown it king. It merely gives the future incident report better evidence.*

II. On Signals, Silence, and Honest Uncertainty

The machine speaks through status, metrics, errors, clocks, and absence. Most outages are not born mute; they are born speaking a dialect no decision was ever bound to. The wise architect does not ask whether there is telemetry. The wise architect asks whether the telemetry can tell the truth without a costume.

21. Fallback must degrade semantics — a fallback that returns the same shape as truth will be mistaken for truth

CRITICAL SEVERITY.

The trap. Helpfulness over honesty — returning a value feels better than returning an error, but a fake value propagates silently. Its familiar disguise: *"Return something rather than nothing"*. Fallbacks may preserve availability, but they must not preserve the illusion of certainty. A fallback value must not return through the same field/type/shape as canonical truth unless explicitly marked as degraded, observed, approximate, stale, or uncertified.

The discipline. Fallback return paths must use a distinct type, wrapper, or status field that the caller can inspect. Returning zero-values or empty strings through the same type as canonical truth is forbidden when the source is a fallback.

22. Authority must express uncertainty — if the owner cannot say 'unknown', callers will turn silence into lies

HIGH SEVERITY.

The trap. Simplicity over precision — returning an empty proto feels clean, but callers can't tell absence from error. Its familiar disguise: *"Empty is a valid response"*. If an owning actor cannot express "unknown," "stale," "degraded," "conflict," "not canonical," or "source unavailable," callers will manufacture certainty from silence.

The discipline. Owner RPCs must return explicit status codes or status fields that distinguish absence from uncertainty. Callers must propagate uncertainty — never silently resolve it to a concrete value.

23. Absence scope must be explicit — 'not found where' is not the same as 'does not exist'

HIGH SEVERITY.

The trap. Confusing one view with all views — a cache miss or replica miss is not proof of global absence. Its familiar disguise: *"Not found means doesn't exist"*. "Not found" is not a fact until the system says where it looked and who owns existence. A cache miss, local file miss, replica miss, index miss, or stale view miss does not prove global absence.

The discipline. Code that acts on "not found" must verify the scope of the lookup before concluding non-existence. A single-replica miss must not trigger deletion, re-creation, or failure escalation that assumes global absence.

24. Errors must not be silent on connection paths — a connection error absorbed into a timeout is an invisible outage

HIGH SEVERITY.

The trap. Generic wrappers feel robust — but grpc.WithBlock() turning a TLS mismatch into "context deadline exceeded" hides the real error for hours. Its familiar disguise: *"Timeouts are normal"*. Connection-establishment code absorbs specific errors (TLS mismatch, wrong key, unreachable host) into generic timeouts or nil values. The caller sees "timeout" or "nil" — the same shape as "slow network" or "not initialized yet." The specific error is lost, and the outage becomes invisible.

The discipline. Connection paths must surface specific errors before falling back to timeouts. Pre-flight checks (TLS validation, auth probe) should run before the generic dial wrapper. Nil handles from failed initialization must be retried, not cached permanently.

25. Every assertion — positive or negative — carries the scope of its truth; aggregation without naming the scope is a lie

CRITICAL SEVERITY.

The trap. A passing check feels universal, an absent record feels final — but every observation lives in a scope (which node, which moment, which tenant); stripping the scope strips the truth. Its familiar disguise: *"OK is OK, missing is missing"*. Both "I see X" and "I don't see X" are scoped observations. The scope is part of the truth. A consumer that reads the observation without the scope can mistake local for global, instance for cluster, moment for forever.

The discipline. Every health/proof/verdict/finding value carries an explicit scope field or typed wrapper covering both positive and negative cases. Aggregation across scopes is allowed only via a function whose name and signature make the aggregation visible.

26. An abstraction that swallows its own failure mode is worse than no abstraction

CRITICAL SEVERITY.

The trap. A helper that returns a sensible default on the rare path feels considerate — callers don't need a branch, the code looks clean. But "sensible default" and "real answer" have the same shape, so every caller now silently treats failure as success. The abstraction has absorbed the failure into the success channel; the bug surfaces three layers downstream as garbage data, and the path back to the swallowed signal is dead. Its familiar disguise: *"If the abstraction returns the same type on success and failure, the caller can carry on"*. An abstraction (helper, resolver, template engine, parser, wrapper) MUST surface its failure mode in a shape distinguishable from success. Returning the same type on "I found it" and "I didn't" makes the caller structurally unable to tell the two apart. The caller's correctness then depends on a runtime assumption the type system cannot enforce.

The discipline. Abstractions whose normal path returns T should make their failure path return one of:

- (T, error) where T's zero value is unambiguous
- (T, bool) where bool is presence — Lampson's "Use hints" pattern, with explicit verification at the call site
- *T — nil-distinguishable from any returned value
- A sum type (Result / Option / sealed interface) where the type system FORBIDS treating failure as success

See also: 21, 22, 42

27. A timeout means ‘I did not hear back in time’ — never ‘the other side failed’

CRITICAL SEVERITY.

The trap. A timeout feels definitive because it’s a hard boundary — the request didn’t complete in the budget. But that’s a statement about YOUR observation, not the world. The other side could be slow but proceeding, finished but with the response in flight, blocked on something you don’t see, or actually down. Turning a four-way uncertainty into a binary “down” is a lie the system then makes decisions on. Its familiar disguise: *“Connection timed out, so the service is down”*. A timeout is the failure of an OBSERVATION to complete within a budget — not the failure of the OBSERVED actor. Tanenbaum’s first fallacy (“the network is reliable”) has a direct corollary: timeouts are not decisive. Code that treats `err == context.DeadlineExceeded` as proof of remote failure makes consequential decisions on incomplete information.

The discipline. Every code path that handles a timeout MUST:

- Distinguish “timeout” from “remote error returned” — they are different signals and merge to the same logged-error type only by mistake.
- Model “unknown” as a third state distinct from “known-up” and “known-down.” Where the consumer can’t represent three-valued state, document the assumption being made.
- When a decision MUST be made on the timeout (because the caller can’t wait forever), either: (a) Pick the SAFE-DEFAULT option that doesn’t presume the other side’s state — usually retry with exponential backoff and a final ABANDON terminal state (see bad_path_must_make_progress). (b) Cross-check with a different observer before acting. (c) Explicitly log “made under timeout uncertainty, may be wrong” so the audit trail names the gamble.
- When acting on a timeout would mutate cluster state (quorum changes, leader transitions, drains), the action MUST require an explicit confirmation from a second source — a different probe, a different observer, or a wait period.

See also: 22, 23, 24, 49

28. A recorded timestamp is the observer’s clock at moment of observation — not when the event actually happened

CRITICAL SEVERITY.

The trap. When a system records an event_time of 13-42-30, the natural reading is “the event occurred at that moment.” But the field was written by a process — the observer — at the moment the observer NOTICED the event. The event may have occurred earlier; the observer may have been delayed; the storage system may have persisted the record at yet another moment. Treating the recorded timestamp as the event’s actual time conflates three distinct moments and corrupts every reconstruction that depends on the difference. Its familiar disguise: *“The record says it happened at T, so it happened at T”*. Every timestamp in a system has a producer — a clock running on a process at the moment of WRITE. The semantic the timestamp carries depends on which clock and what the writer was observing. There are at least four distinct moments the same record could mean:

The discipline. Every timestamp field in a schema or message must declare:

- WHICH CLOCK produced it (wall, monotonic, logical).

- WHICH OBSERVER recorded it (process, role, layer).
- WHAT MOMENT it captures (event, observation, transport, storage).
- WHAT THE CONSUMER may infer from it (causality only, bounded ordering with skew tolerance, observational-only).

See also: 2, 8, 25, 29

29. Comparing timestamps from two nodes without a skew bound is structurally unsound — Spanner's TrueTime made this explicit by returning uncertainty intervals, not points

CRITICAL SEVERITY.

The trap. NTP keeps clocks within a few milliseconds of UTC, so comparing two NTP-synced timestamps feels like comparing two ground-truth values. But NTP's accuracy guarantee is statistical, not bounded — a node mid-sync can be off by hundreds of milliseconds; a node post-leap-second can be off by a full second; a node with a broken NTP daemon can be off by minutes. Code that compares timestamps without a documented skew bound is making decisions on an uncertainty interval it doesn't know exists. Its familiar disguise: *"Both nodes use NTP, so their clocks agree closely enough"*. Google's Spanner introduced TrueTime as the operationalization of clock-uncertainty thinking. TrueTime returns not a moment but an INTERVAL — TT.now() = [earliest, latest]. Any operation that depends on "is A before B?" must compare the intervals, not the midpoints; if the intervals overlap, the question is structurally unanswerable without waiting for the uncertainty to clear.

The discipline. Code that compares timestamps across processes or nodes MUST satisfy one of three contracts:

- SKEW-BOUNDED: the maximum drift between the two clocks is known (via NTP stratum, lease keepalive, or PTP) and the comparison logic accounts for it. Acceptable when the bound is tight relative to the decision's sensitivity.

See also: 8, 22, 28, 45

30. When auth flows through a proxy, the downstream service must know WHO originated the request — not WHO proxied it

CRITICAL SEVERITY.

The trap. A gateway or sidecar that handles authentication feels like a natural boundary — once past the gateway, the request is trusted. But the gateway's identity is not the user's identity, and downstream services that authorize against the proxy's principal instead of the originating principal grant access to anyone the gateway proxies for. This is the confused deputy problem under a new name; it has been written about since the 1980s and keeps recurring because the proxy boundary FEELS like an authorization boundary, but the principal identity must traverse it intact. Its familiar disguise: *"The gateway authorized the request, so the service can trust the gateway's principal"*. Hardy's 1988 description of the "confused deputy" problem named a failure shape that the security community has been rediscovering ever since: a privileged intermediary, asked to perform an action on behalf of a less-privileged principal, performs the action with its OWN authority rather than the principal's. The principal's identity was lost in transit; the action succeeds with the intermediary's broader access; security is silently breached.

The discipline. Every RPC handler that authorizes the caller must verify two identities:

- The TRANSPORT identity — the mTLS peer cert that established the connection. This identifies the IMMEDIATE caller (gateway, sidecar, peer service).
- The PRINCIPAL identity — the originating user, AI service, scheduled task, or operator on whose behalf the request was made. This identifies WHO the action authorizes against.

See also: 2, 9, 25, 45

31. The granularity and freshness of emitted signal must match the granularity and freshness of the decisions that depend on it

CRITICAL SEVERITY.

The trap. Observability feels like a fixed property of a system — you set up Prometheus, you have metrics. But the cadence at which signals are emitted determines the cadence at which decisions can be made on them. Emit hourly and you cannot react sub-hourly; emit per-millisecond and you drown in volume the consumer cannot process. The cadence is not a back-end choice; it is a contract between the producer and every decision that depends on the data. Its familiar disguise: *"Emit one heartbeat per minute and use it to drive sub-second routing"*. The granularity at which a system emits signals and the freshness of those signals must match the granularity and freshness of the DECISIONS that consume them. Mismatch produces one of two failure modes:

The discipline. Every signal emitted by the system must carry an implicit or explicit cadence contract. Producers state how often they emit and at what granularity; consumers state what decisions they make on the signal and the freshness they require.

See also: 7, 33, 44, 52

32. Alerts that fire on numerical thresholds produce noise during normal spikes and silence during creative failures — alerts should name failed invariants

CRITICAL SEVERITY.

The trap. A threshold alert (CPU > 90, error rate > 5%, latency > 200ms) feels like a defined condition — when the number crosses the line, something is wrong. But the threshold encodes only ONE failure shape — the one whose symptom is that specific number. Real failures take infinite shapes; healthy load spikes produce the same number. The threshold confuses symptom with cause and produces noise during normal operation and silence during anything novel. Its familiar disguise: *"Alert when CPU is over 90%"*. An alerting system based on numerical thresholds has two structural failure modes: FALSE POSITIVES during normal operation. Load spikes, legitimate batch jobs, planned maintenance, and seasonal variation all cross the same thresholds healthy systems sometimes hit. Operators learn to ignore the alerts because the alerts are usually wrong.

The discipline. Alerting policy in any new code:

- NAME the invariant the alert protects ("availability SLO held over the rolling 30-day window", "every required node converges within the release deadline", "RBAC's own health probe must succeed").
- The metric is the EVIDENCE supporting the invariant check, not the alert condition itself. Alert fires when the invariant FAILS, not when the metric crosses.

- The alert payload names BOTH the invariant and the evidence, so the operator's first question — "what promise was broken?" — has an answer in the alert message.
- If you cannot name the invariant the alert protects, the alert should not exist. Threshold-based alerting without an invariant is operator noise.

See also: 25, 31, 33, 42

33. Aggregation summarizes; aggregation also destroys — the individual events that contained the actionable signal are gone

CRITICAL SEVERITY.

The trap. A summary statistic feels like a complete answer — p99 latency tells you the slow tail, error rate tells you the failure rate, throughput tells you the load. But aggregation discards every property of the individual events that did not contribute to the summary number. The p99 tells you SOMETHING is slow; the outlier requests tell you WHAT. If only the aggregate survives, the operator has a number to watch but not a problem to fix. Its familiar disguise: *"We have p99 latency, we know what is slow"*. Aggregation is information loss by design. A histogram preserves shape but loses identities of contributing events. A counter preserves total but loses per-event attribution. A gauge preserves current value but loses history. The information loss is the point — aggregates are cheap to store and fast to query precisely because they are smaller than the events that produced them.

The discipline. For every metric emitted, ask:

- What decision does this metric drive?
- Does the decision need to know WHICH events contributed, or just HOW MANY?
- If WHICH, the metric is insufficient — the system must also emit identifying events (structured logs, findings, traces) with the dimensions the decision needs.
- The metric becomes a SUMMARY of the events; the events are the ground truth for any investigation.

See also: 25, 31, 32, 44

34. Yield (queries completed) and harvest (data per answer) are two separate axes of availability — preserving one says nothing about the other

CRITICAL SEVERITY.

The trap. A successful response feels like the binary answer to "is the system available." But availability has two distinct dimensions — did the query complete (yield) and was the answer complete (harvest). A search that returns three results instead of thirty has 100 percent yield and 10 percent harvest; a partition that drops half the requests has 50 percent yield and unknown harvest. Conflating them lets the system report green while quietly serving impoverished answers, and lets operators report uptime while users see degraded data. Its familiar disguise: *"The system is available because the request returned"*. Eric Brewer (IEEE 2001) introduced this distinction in "Lessons from Giant-Scale Services" as the core insight that uptime alone is too coarse for systems that must stay available under fault.

The discipline. Every component that produces aggregate or federated answers must:

- MEASURE both dimensions. Yield is request outcome accounting; harvest is "what fraction of the underlying universe contributed."

- SURFACE harvest to consumers. An answer with reduced harvest must carry the reduction in its envelope — a “completeness” or “data_available” field that consumers can read to know they got partial data.
- DECIDE which dimension to preserve under fault. Replicated subsystems lean toward harvest preservation (refuse some queries, answer the rest completely). Partitioned subsystems lean toward yield preservation (answer every query, with whatever fragments are available). The choice must match the workload’s tolerance for each kind of degradation.
- ALERT on each dimension separately. A drop in yield is one signal; a drop in harvest is a different one. A single “availability” alert that aggregates both is structurally wrong.

See also: 23, 25, 33, 81

35. ‘Nothing found’ is not a finding — the observer must distinguish ‘searched and verified empty’ from ‘never searched’

HIGH SEVERITY.

The trap. An empty result feels like a clean bill of health — but every query that returns zero results is ambiguous between “I checked everything relevant and found nothing” and “I have no coverage of this area at all.” The consumer treats both as safety; only the first one is. Its familiar disguise: *“No news is good news”*. When an observation system returns an empty result, it produces a signal that has the SHAPE of safety but may carry the MEANING of ignorance. The consumer cannot distinguish these without explicit coverage metadata from the observer.

The discipline. Every query system that can return an empty result must return a coverage attestation alongside it — one of: - a coverage class field: {attested, partial, uncharted} - a surveyed_at timestamp (nil = never surveyed) - a coverage_scope describing what was actually searched.

See also: 22, 23, 25, 31

36. An event is a notification to go ask the owner — never the authoritative state itself

HIGH SEVERITY.

The trap. A delivered event feels like the fact — but the message is a hint, and messages are lost, duplicated, delayed, and reordered. Its familiar disguise: *“The event said it changed, so I’ll mutate local truth from the event payload”*. Events may wake reconcilers, but events must not be the source of authoritative state. An event means “something changed, go ask the owner”, never “blindly apply this delta to local truth”.

The discipline. Event handlers must re-read authoritative owner state before acting, not trust the payload as truth. Convergence must not depend on a complete, ordered event stream. Repair must never be driven by an unordered event stream alone.

See also: 1, 42, 49

37. Failure is classified into actionable states, not collapsed to FAILED

HIGH SEVERITY.

The trap. A single FAILED feels simple — but it erases the one thing the next actor needs, which is what to do about it. Its familiar disguise: *“one FAILED state for every kind of failure”*. FAILED is too dumb. Classify: FAILED_RETRYABLE, FAILED_TERMINAL, FAILED_DEPENDENCY_UNAVAILABLE,

FAILED_AUTHORITY_REJECTED, FAILED_TIMEOUT, FAILED_INVARIANT_VIOLATION, COMPENSATION_REQUIRED, ROLLBACK_REQUIRED, UNKNOWN_NEEDS_RECONCILE. Each failure state implies its allowed next actions.

The discipline. Failure states are a classified enum; each maps to permitted next transitions. A bare boolean/failed collapse is a violation.

See also: 22, 24

38. Observations are evidence, not transitions — a probe never directly mutates state

CRITICAL SEVERITY.

The trap. A failed probe feels like a verdict — but it is one observation, and observations must pass through reconciliation. Its familiar disguise: *"probe failed -> state = REMOVED"*. A probe, heartbeat, log, metric, or doctor finding is evidence. It must not directly become state without reconciliation: EventProbeFailed -> reconciler evaluates threshold/current generation/authority -> transition HEALTHY -> SUSPECT or DEGRADED. Observation informs; the owner transitions.

The discipline. No observation handler sets terminal/membership state directly. It appends an evidence event that a reconciler evaluates.

See also: 15, 36

39. Control-plane availability, data-plane availability, durability, and convergence are distinct dimensions and must be reported separately

HIGH SEVERITY.

The trap. A single health light feels reassuring — but a collapsed control plane and intact data (or vice versa) are different truths that a single verdict hides. Its familiar disguise: *"One 'cluster healthy' verdict covers everything"*. A platform must separately report control-plane availability, data-plane availability, durability, and convergence. One can be healthy while another is degraded: etcd may be unavailable while Scylla data remains intact; MinIO may serve some reads while healing or under-protected; Scylla may be running while RF/ownership is unsafe; a node-agent may be healthy while the cluster topology is not.

The discipline. Status output must name which dimension is healthy or degraded rather than collapsing them into one signal. A degraded data plane must not be masked by a healthy control plane, and neither must be inferred from the other. Sibling of meta.harvest_and_yield_are_distinct_availability_dimensions on a different axis.

See also: 18, 25, 34

> *Field note — A dashboard with forty-seven green lights may still be a beautifully illuminated absence of proof.*

III. On Time, State, and the Long March of Work

Every write is a promise, and a promise no one is appointed to keep is a blockage wearing the face of progress. The long-running campaign is not lost to the enemy you see. It is lost to the half-finished state that looks finished, the retry that becomes a siege aimed inward, and the intermediate step that answers 'yes' when asked if it is done.

40. Every write is a promise — an unfinished promise is a blockage

CRITICAL SEVERITY.

The trap. Fire and forget feels efficient — but a blocking record without a cleanup path becomes a permanent stall. Its familiar disguise: *"Write and move on"*. When code writes a state record (etcd key, Scylla row, convergence record, lock, status entry), it creates an obligation. Either the same actor completes the lifecycle, or there is an explicit sweep that clears stale records, or the record has a TTL. Half-written state that nobody owns the cleanup for becomes a permanent stall.

The discipline. Every function that writes a blocking record must have a corresponding cleanup path (explicit delete, sweep, or TTL). Code review must verify the cleanup path exists before approving the write.

41. Half-done must never look done — intermediate state must not satisfy completeness predicates

CRITICAL SEVERITY.

The trap. Premature optimization of predicates — checking artifact_state without manifest_json feels sufficient, but a partial write satisfies the single-field check. Its familiar disguise: *"One field is enough to check"*. When a multi-step operation is interrupted, the intermediate state must be distinguishable from the terminal state. If a skip/completeness predicate cannot tell the difference between "done" and "half-done," every interruption becomes permanent.

The discipline. Completeness predicates must check ALL fields that a complete operation would have written. A single-field check on a multi-field write is a violation. State machines must have explicit intermediate states that skip predicates reject.

42. Silence is not a valid response to the unexpected — unhandled cases must fail closed

HIGH SEVERITY.

The trap. A no-op default feels harmless — but a silently skipped switch case means the system stops making progress without anyone noticing. Its familiar disguise: *"Default is safe"*. When a switch/dispatch does not match, the default must be an explicit error or escalation — not a no-op. Silent discard of an unrecognized case is how a system stops making progress without anyone noticing.

The discipline. Switch statements on state/status enums in dispatch or state-machine code must have a default case that logs the unexpected value. Silent no-op defaults in release pipeline routing are forbidden.

43. Failure response must contract, not amplify — retry, re-enqueue, and re-emit must be bounded, or the response becomes the outage

CRITICAL SEVERITY.

The trap. Persistence feels like resilience — but unbounded retry under failure consumes the resources that recovery needs. Its familiar disguise: *"Retry harder"*. When something fails, the system's natural response is to retry, re-enqueue, re-emit, or respawn. If that response is unbounded, it amplifies the original failure into a cascade. The system doesn't die from the error — it dies from its own reaction to the error.

The discipline. Every retry path must have a circuit breaker or bounded backoff. Fire-and-forget goroutines on error paths are forbidden — each one must be tracked and bounded. Re-enqueue after failure must be staggered, not simultaneous. Event emission must be deduplicated against a delta cache.

44. Diagnostic output must be bounded — one error must not become N records, or the diagnosis becomes a harder failure than the disease

CRITICAL SEVERITY.

The trap. Observability feels always good — but unbounded logging under sustained failure fills the disk, and disk full kills services that were perfectly healthy. Its familiar disguise: *"Log everything for debugging"*. When an error repeats, the system's diagnostic output (logs, events, convergence records, audit entries) grows proportionally. If the output is unbounded, a sustained error fills a shared resource — disk, etcd quota, memory, event bus. The resource exhaustion is a HARDER failure than the original error: a login bug is fixable, but a full disk kills everything including the tools you'd use to fix the login bug.

The discipline. Error-path logging must use rate-limiting (log first occurrence + count, not every occurrence). Event emission must be deduplicated against a delta cache. Convergence records must overwrite (deterministic action ID), not accumulate. Shared resources (disk, etcd) must have usage alerts that fire before exhaustion.

45. Every binding carries 'I checked this then' — when 'now' is different from 'then', the binding is a phantom unless re-validated

CRITICAL SEVERITY.

The trap. A decision feels durable once made — but the evidence the decision was bound to may have moved, and the binding without its evidence is a ghost that authorizes the wrong present. Its familiar disguise: *"If it was approved, it's approved"*. When an authority issues an approval, emits a proof, or sets a bootstrap marker, it does so against evidence that was current at that moment. The binding from authority to consumer carries an implicit "I checked this then." Time passes — seconds, minutes, a failover, a restart. The "then" world the binding was made against may no longer exist; the binding persists as if it does.

The discipline. Every binding from authority to consumer must include an evidence digest or generation number that the consumer re-validates at consumption time. Bindings whose validity depends on phase or time must carry their own freshness check; in-memory-only bindings across actions that can span a restart or failover are forbidden. Bootstrap markers must be paired with a clearing path (event or level-triggered sweep) OR a freshness check at every consumer that respects them.

46. Intent commits before action, retry is the only response to failure — no alternative paths once committed

CRITICAL SEVERITY.

The trap. Computing the actions and executing them inline feels efficient, and falling back to a different path on failure feels resilient — but a failure between step 1 and step N leaves the world half-changed with no resume point, and an alternative path leaves the audit record claiming you did X when you actually did Y. Its familiar disguise: *"Just run the steps; if X fails, try Y instead"*. Two halves of one contract: Commit-first half: A state-changing operation that spans more than one observable side effect MUST be committed to a durable coordinator BEFORE any side effect runs. The coordinator owns: (a) the intent record (what should happen), (b) the per-step progress (where we are), (c) the terminal-state guarantee (every started intent reaches SUCCEEDED or FAILED), and (d) the resume contract (a new process picks up where the old one left off, idempotently).

The discipline. Every state-changing operation that spans more than a single etcd transaction MUST flow through the workflow service. Direct CLI → etcd writes are forbidden for any change that requires multiple coordinated side effects. MCP tools that mutate state must do so via plan → validate → approve → execute (which dispatches a workflow), never via direct RPC to a state-changing endpoint.

47. Code that crystallizes before its true shape is understood becomes cement around its bug

CRITICAL SEVERITY.

The trap. Consolidation feels like maturity — once you see the pattern, codify it. But codifying *too early*, before the universe is known, freezes the wrong shape. The consolidated form then gets imported, depended on, and trusted, until changing it requires touching every dependent. The bug becomes structural, not local; fixing it requires architecture, not edits. Its familiar disguise: *"Now that we have N examples, let me write the canonical N-entry list"*. Lampson's "Plan to throw one away" (§3.3, 1983) is usually read as advice about implementation strategy: build the first version expecting to discard it. The deeper observation is about TIMING of canonicalization. A hardcoded list, a sealed type, an exhaustive switch, a frozen interface — each of these is a commitment that the current understanding is complete. When the commitment is made before understanding is complete, every subsequent consumer inherits the frozen wrong shape.

The discipline. Before consolidating an emerging shape into a canonical form, ask:

- Has the set stopped growing? (Three months without a new member? Three users? Three failure modes?)
- Is the consolidation derivable from a still-evolving source, or is the consolidation itself the source?
- If a new member appears tomorrow, what changes? Is it a one-line addition, or does it require touching every consumer?
- Does the canonical form have an "I don't know yet" slot, or does it force every member to commit to all dimensions now?

See also: 2, 5

48. Failure response must move the system toward a terminal state, not freeze it indefinitely

CRITICAL SEVERITY.

The trap. Retry feels safe — if you don't know whether a transient failure will clear, why not keep trying? But infinite retry without progression turns the system into a CPU heater. The failure isn't getting worse, but it isn't getting better either, and from the operator's view the system is "still trying" forever. Stuck-with-no-progression is its own failure mode, distinct from amplification, and equally fatal. Its familiar disguise: *"Retry forever; eventually it'll work"*. Lampson's "Separate normal and worst case" (§2.1, 1983) usually gets quoted for the speed half — "the normal case must be fast." The fault-tolerance half — "the worst case must make some progress" — is the load-bearing one. A failure-handling path that loops without progression is indistinguishable from a hang; from the operator's view the system has frozen. The CPU is busy, the logs are flowing, the workflow runs are dispatching — and nothing is changing. This is its own failure mode, distinct from amplification.

The discipline. Every retry, defer, backoff, circuit-breaker, or wait loop must answer:

- What's the upper bound? (Attempts, wall-clock, distinct failure shapes?)
- What terminal state is reached when the bound is hit?
- Who sees the terminal state — operator, ticket, alert, workflow recorder?
- Is the terminal state DIFFERENT from the looping state? (If the system goes from "trying" to "trying" forever, the principle is violated even if the rate is bounded.)

See also: 40, 41, 43

49. Every operation that can be retried, replayed, or re-dispatched MUST be idempotent — there is no 'might be retried'

CRITICAL SEVERITY.

The trap. Idempotence reads like a property — "this operation happens to be safe to run twice." That framing makes it optional, something you adopt once a problem shows up. But in any system with retry, restart, dispatch deduplication, workflow resume, or message redelivery, every operation will eventually run multiple times. Whether the operation is idempotent determines whether the system is correct or corrupted. Its familiar disguise: *"We'll make it idempotent later if we need to"*. In any system with retry budgets, workflow resume, dispatch deduplication, restart safety, hot-standby failover, or message replay, EVERY operation is potentially executed more than once. This is not the exception; it is the design surface. Code MUST be written assuming multi-execution is the normal case.

The discipline. Every step in a workflow, every handler an actor exposes, every retry-capable RPC must satisfy idempotence one of two ways:

See also: 27, 40, 46

50. 'Wait 30 seconds' and 'wait until 13:42:30' mean different things across restarts, pauses, and NTP corrections — the choice is semantic

CRITICAL SEVERITY.

The trap. A 30-second backoff and a 13:42:30 deadline computed as `now + 30s` look equivalent at the moment they're created. But they age differently — a duration restarts on resume, a deadline persists; a duration is immune to NTP correction, a deadline isn't; a duration depends on knowing "when did I start," a deadline depends on knowing "what time is it now." Code that stores one and interprets it as the other will be silently wrong at exactly the moment a restart or correction occurs. Its familiar disguise: *"Duration and*

deadline are different ways of saying the same thing". Time-bound state in a system can be encoded one of two ways: DURATION: "wait N units after time T_0." Stored as the unit count; the meaning depends on knowing when T_0 was. Restart- and-resume implies the duration counter restarts (the process forgot its T_0). NTP corrections don't shift the meaning (because the duration is relative to a remembered point).

The discipline. Every time-bound field in a schema, RPC, or workflow input must carry an unambiguous name:

- `*_duration_seconds` or `*_after_ms` for DURATION semantics.
- `*_until_unix` or `*_deadline_ms` for DEADLINE semantics.
- Both, when the code needs to be robust to both restart and NTP correction.

See also: 7, 28, 40, 48

51. An authorization decision is bounded by the moment it was made — the underlying authority can change before the action is taken

CRITICAL SEVERITY.

The trap. An authorization check feels like a transaction — it succeeds, and now you know the principal is allowed. But the check measured the state of permissions at the moment of measurement, not the state for the lifetime of the operation. Role bindings change, tokens get revoked, certificates expire, accounts get disabled. Code that treats a successful check as a permanent green light for the duration of an operation creates a window where the operation continues with permissions that have been revoked. Its familiar disguise: *"We checked permissions at the start, so the long-running operation is good for the whole run"*. Saltzer and Schroeder's third principle — complete mediation — states that every access to every object must be checked for authority, every time. Operationalized in modern systems: any authorization decision is a SNAPSHOT of permission state at the moment of the check; the snapshot becomes stale the instant the next change to the underlying authority occurs. The TOCTOU window is between the snapshot and the action.

The discipline. Authorization caching is permitted with three conditions:

- EXPLICIT TTL — the cache entry has a maximum age past which it MUST be revalidated.
- INVALIDATION SUBSCRIPTION — when the underlying authority changes (role binding update, token revocation, cert rotation), the cache is invalidated synchronously.
- ACTION-BOUNDARY REVALIDATION — for destructive or irreversible operations, the cache is bypassed and a fresh check is performed regardless of TTL.

See also: 2, 7, 9, 45

52. A control loop running slower than the drift it is meant to correct will oscillate or diverge — never converge

CRITICAL SEVERITY.

The trap. A reconciler that runs periodically feels like it eventually catches up. If something drifts, the next tick will detect and correct. But control theory has a hard result on this — if the disturbance rate exceeds the correction rate, the system either oscillates around the target without converging, or diverges away from it. The reconciliation cadence is not a configuration choice; it is a stability condition. Its familiar disguise: *"We reconcile every 5 minutes and that should be enough"*. Classical control theory: for a feedback system to

converge on a target, the loop's response time must be FASTER than the disturbance rate of the system being controlled. If the disturbance arrives faster than the loop can correct, the system either:

The discipline. For every reconciler, scheduler, or control loop:

- Identify the DISTURBANCE RATE — how often the controlled state changes in normal operation.
- Identify the LOOP FREQUENCY — how often the loop executes.
- Verify the loop frequency is AT LEAST 2x the disturbance rate (Nyquist condition).
- Where the loop must be slower (cost, fanout, coordination), DOCUMENT the maximum disturbance rate the loop can track and surface as a finding when the system exceeds it.

See also: 31, 40, 43, 48

53. Atomic single-write of state is necessary but not sufficient — growing or large state needs append-only logs alongside the checkpoint

CRITICAL SEVERITY.

The trap. Atomic whole-state writes feel safe — either the new state is fully there or the old state is, no torn writes, no partial corruption. But "atomic" is paid for in throughput, and the cost is proportional to the total state size, not to the size of the change. By the time the system is in production with non-trivial state, the atomic write is seconds or minutes long, and you've structurally locked yourself out of partial recovery — every change requires reading and writing everything. Its familiar disguise: *"We just rewrite the whole file atomically on every update"*. Eden (Black 1985) discovered that its checkpoint primitive — an atomic write of the entire object state to disk — was "conceptually very simple, and fulfills the goal of ensuring that the state of an object can survive a crash," but "neither a primitive nor a solution." For many objects, atomic-whole-write was both too slow and structurally wrong, and the team eventually concluded that checkpoint should be combined with append-only logging for partial updates.

The discipline. For every persistent state in the system, classify it as one of:

See also: 40, 46, 48

54. Users will not adopt a new system that requires complete replacement of their existing one — adoption demands migration paths, not capability superiority

CRITICAL SEVERITY.

The trap. A clean new architecture deserves clean adoption — let the new system have its own shape, let users rewrite their workflows to fit. But the user has existing systems running, existing tooling, existing operational knowledge. Forcing them to rewrite is a tax they will not pay regardless of how superior the new architecture is. Eden explicitly named this — "it was naive for us to expect that people would clamour to use the Eden system in their daily work" — and survived only because they ran on top of UNIX and let users migrate incrementally. Its familiar disguise: *"Our architecture is superior; users will see that and convert"*. Eden (Black 1985 §3.4.4) is unusually candid about the adoption problem. The Eden team had an architecturally superior system; users did not adopt it on its merits. The team's diagnosis — "It is easy to underestimate the amount of effort required to produce a system that other people would want to use for their daily work in the way they use UNIX or TOPS-20. Lampson and Sturgis have claimed that a kernel is ten per cent of an operating system" — was that capability was 10% of the adoption problem. The other 90% was migration path.

The discipline. For every new feature, capability, or design choice that affects how users interact with Globular:

- Identify the EXISTING tool, format, or workflow the user has in production today (systemd, raw scripts, Prometheus, JWT, OAuth, kubernetes, docker compose).
- Default to a design that allows the existing tool to keep working unchanged. The Globular-native form is additive, not replacing.
- Document the migration path explicitly. “User has existing X; here is how X works inside Globular with zero changes; here is how X can incrementally gain Globular-native behavior.”
- The migration path is a first-class feature, not a post-hoc workaround.

See also: 5, 47, 48

55. For systems that change frequently, repair time is much easier to improve than failure rate — and has equal impact on uptime

CRITICAL SEVERITY.

The trap. Making failures rarer feels like the right answer to “the system fell over” — fix the bugs, harden the code, eliminate the failure modes. But MTBF is hard to measure (you need weeks of realistic load to validate one data point) and easy to confuse yourself about (you fixed the failures you noticed, not the ones you did not). MTTR is measurable in minutes, improvable through tooling, and stable across feature additions. For systems that change frequently — every Globular service ships releases regularly — MTBF stability is a fiction. MTTR is what actually moves availability. Its familiar disguise: *“We will get availability by making failures rarer”*. Brewer (IEEE 2001): “We can improve uptime either by reducing the frequency of failures or reducing the time to fix them. Although the former is more pleasing aesthetically, the latter is much easier to accomplish with evolving systems.” The arithmetic is symmetric (uptime = (MTBF - MTTR) / MTBF), but the engineering effort is not.

The discipline. For every reliability investment in the codebase:

- CLASSIFY the investment as MTBF (preventing failure) or MTTR (speeding recovery).
- For MTBF work, name the failure mode being prevented. Mode-specific MTBF work compounds usefully; speculative MTBF work does not.
- For MTTR work, measure the recovery time in the before and after state. The measurement is cheap.
- Where the choice exists, prefer MTTR work for recoverable failure classes; reserve MTBF for catastrophic classes.

See also: 31, 48, 52, 82

56. Saturation is the normal operating mode for production systems — graceful degradation is the explicit response, not an emergency fallback

CRITICAL SEVERITY.

The trap. Saturation feels like a rare emergency — something the system enters only when capacity planning failed. So degradation behavior is treated as last-resort, often unwritten, sometimes never tested. But Brewer’s measurements showed peak-to-average ratios of 1.6 to 6 in production traffic; provisioning for peak would mean running at 17 to 60 percent average utilization, which is uneconomical. So real systems run hot, and

saturation is regular not rare. Treating it as an emergency leaves the response to that emergency unpracticed and undocumented. Its familiar disguise: *"Provision for peak load and saturation will not happen"*. Brewer's exact observation: "the peak-to-average ratio for giant-scale systems seems to be in the range of 1.6 to 6, which can make it expensive to build out capacity well above the (normal) peak." Provisioning for peak with comfortable headroom means most resources sit idle most of the time. So production systems run at high utilization, and saturation events are not rare — they are expected.

The discipline. For every subsystem that serves requests under variable load:

- IDENTIFY the saturation threshold — utilization or queue depth or RPC rate above which the subsystem's latency or error rate sharply degrades.
- DESIGN an explicit degradation mode for the regime above the threshold. Three Brewer-style options are available — admission control, dynamic content reduction, or hybrid.
- EXPOSE the degradation mode via metrics. Operators should be able to see "the system is currently in degraded mode" as a first-class signal.
- EXERCISE the degradation mode under controlled load. A degradation mode that has never been exercised will not work when needed.

See also: 34, 43, 48, 84

57. Every install action has a matching uninstall owner — cleanup removes only what a package/profile/generation owns

HIGH SEVERITY.

The trap. Removal feels safe because it's 'just cleanup' — but uninstall without ownership is archaeology with a shovel of guesswork. Its familiar disguise: *"Tear down whatever looks related and hope nothing else needed it"*. Every install action must have a matching uninstall/rollback owner, and cleanup must remove only artifacts owned by that package, profile, and generation. Installs create system links, service files, firewall rules, binaries, PATH changes, DNS/resolv changes, and dependency files; if ownership is not recorded, uninstall becomes guesswork and either leaves stale state (the sticky-package / stale-keepalived-config class) or removes something still in use.

The discipline. Maintain a durable install ledger: each artifact records its package owner, profile owner, generation, rollback command, and uninstall verification. Cleanup consults the ledger and removes only owned artifacts. No blind pattern-based teardown of shared system state without an ownership check.

See also: 13, 40

58. Recovery mode is explicit, evidence-gated, and strictly narrower than normal mode — emergency shortcuts must never become normal paths

CRITICAL SEVERITY.

The trap. An emergency feels like permission to skip authority — but an unmarked shortcut becomes the default path and then the next outage. Its familiar disguise: *"The system is broken, so bypass the owner and write the fix directly"*. Any code path that bypasses normal authority because the system is broken must be explicitly marked recovery mode, require evidence, and allow FEWER actions than normal mode — never more. Distributed platforms always need emergency tools; the danger is when emergency shortcuts quietly become the normal path.

The discipline. Forbidden as silent fallbacks: “if controller unavailable, write etcd directly”; “if desired state missing, infer from local config”; “if install half-failed, retry without ownership”. A recovery path must be an explicit recovery transaction with a bounded target, dry-run evidence, an audit log, and a reconciliation back to owner truth once recovered.

See also: 9, 15, 79

59. Workflow state is a declared machine state, not status text

HIGH SEVERITY.

The trap. A free-form status string feels descriptive — but a string has no allowed transitions, no owner, and no terminal classification. Its familiar disguise: *“status = ‘processing’”*. A workflow state must be a declared machine state, never a free-form string (running, done, failed, pending). A useful state declares: allowed incoming transitions, allowed outgoing transitions, owner/authority, retry behavior, terminal/non-terminal classification, and the evidence required to enter it. STATE_APPLYING_PACKAGE / STATE_WAITING_AGENT_ACK / STATE_FAILED_RETRYABLE — not status=”processing”.

The discipline. Workflow state must be a closed enum of declared states, each with a transition table. A string status field used as workflow state is a violation. The empty string is not a state (see the “” -> “WAITING” invalid-transition class).

See also: 4, 60

60. Every transition is explicit — no hidden state jump

HIGH SEVERITY.

The trap. Setting a boolean feels like progress — but an implicit jump can’t be asked ‘where can this state go next?’. Its familiar disguise: *“if err == nil { done = true }”*. State must change only through named transitions with a reason: transition(APPLYING -> VERIFYING, reason=package_applied). No boolean flags that silently imply a state change. Explicit transitions make the graph queryable (“where can this state go?”) instead of a swamp of booleans.

The discipline. All state change flows through a transition() call recording (from, to, reason). Scattered boolean mutation of workflow status is a violation.

See also: 42, 59

61. State transition and its evidence event must be one atomic durable write

CRITICAL SEVERITY.

The trap. Writing state then publishing feels sequential and fine — until a crash lands between them. Its familiar disguise: *“state = COMMITTED; publish committed event (crash between splits reality)”*. If the workflow moves A -> B, the evidence event that caused the move must be persisted in the SAME durable transaction as the state update. Notification is published only after commit. A crash between state and history must never leave state advanced without its cause recorded.

The discipline. transaction { append EventPackageVerified; update state VERIFYING -> COMMITTED } then publish. State update and history append in separate transactions is a violation.

See also: 46, 62

62. Workflow history is append-only — you append corrections, never rewrite the past

HIGH SEVERITY.

The trap. Editing the failed step to say success feels like fixing it — it destroys the audit and replay trail. Its familiar disguise: *"change old event result from failed to success"*. Never rewrite what happened. Append correction, compensation, retry, or migration events: EventStepFailed -> EventRetryRequested -> EventStepSucceeded. Append-only history is what makes audit, replay, doctor diagnosis, and repair possible.

The discipline. Deleting or mutating a past event is a violation. Corrections are new appended events referencing the prior one.

See also: 53, 63

63. Current state is a rebuildable projection of the history log

HIGH SEVERITY.

The trap. The current-state row feels like the source — but if you can't rebuild it from history, the history is decoration. Its familiar disguise: *"current_state table is truth and history is optional logs"*. The durable event log is the source; current state is a cache/projection (history -> reducer -> current_state). Keep current state for performance, but if you delete it the system MUST rebuild it from events.

The discipline. There must exist a reducer that rebuilds current state from history. If the current-state store cannot be dropped and regenerated, authority has leaked into the projection.

See also: 1, 122

64. Transition guards are named contracts, not scattered if-statements

HIGH SEVERITY.

The trap. Inline conditions feel sufficient — until an agent 'simplifies' the scattered logic into a bug nugget. Its familiar disguise: *"allow the transition if a pile of inline booleans happen to be true"*. Every important transition names why it is allowed. JOIN_PENDING -> ADMITTED guard: signed_join_plan_valid, node_identity_matches, controller_generation_current, no_existing_active_node_with_same_id. Named guards keep the precondition set inspectable and prevent accidental simplification into a bug.

The discipline. Guards are declared, named contracts attached to the transition — not anonymous inline conditionals.

See also: 60, 101

65. Every command/callback at a boundary is idempotent — any external signal can arrive twice

CRITICAL SEVERITY.

The trap. Assuming exactly-once delivery feels reasonable — HTTP retries, RPC-timeout-after-success, and broker redelivery make it false. Its familiar disguise: *"callback InstallDone causes COMMITTED every time it arrives"*. Any external signal (HTTP retry, agent callback, timer, broker event, RPC timeout after success) can arrive twice. Every command/callback carries workflow_id, operation_id/transition_id, expected state/generation, and an idempotency key. A duplicate returns the previous result, it does not re-apply the transition.

The discipline. Each transition is keyed by operation_id and applied at most once. Duplicate delivery is detected and returns the recorded outcome.

See also: 49, 68

66. Mutating commands carry the generation they observed — stale actors cannot move state

HIGH SEVERITY.

The trap. A command feels valid because it arrived — but it may reflect truth from three generations ago. Its familiar disguise: *"Cancel(workflow_id) with no observed generation"*. Every mutating command includes the state/generation it observed: Cancel(workflow_id, expected_generation=42). If the workflow is already at generation 45, the command is stale — reject or re-evaluate. Latest is a number, not a feeling.

The discipline. A mutating command without an expected generation, or with a stale one, is rejected.

See also: 3, 8

67. Terminal states are final except through an explicit recovery contract

HIGH SEVERITY.

The trap. Reopening a terminal workflow inline feels convenient — it erases the terminality that other actors rely on. Its familiar disguise: *"FAILED_TERMINAL -> RUNNING as a normal transition"*. Terminal means no normal transition leaves it (COMMITTED, CANCELLED, FAILED_TERMINAL, ROLLED_BACK). Reopening requires an explicit recovery/migration workflow, never a normal state jump: FAILED_TERMINAL -> RECOVERY_REQUESTED only via an operator recovery contract.

The discipline. No transition table entry leaves a terminal state except one gated by a declared recovery contract.

See also: 48, 58

68. External side effects cross an outbox/activity boundary, never interleave with state

CRITICAL SEVERITY.

The trap. Doing the effect then recording it feels direct — a crash between them creates an effect with no record, or a record with no effect. Its familiar disguise: *"call external service; update workflow state (crash between = ghost effect)"*. Workflow state and outside-world effects must not be interleaved casually. Schedule the effect through a durable outbox in the same transaction as the state change; a worker performs it with an operation_id, and an idempotent callback completes the activity: transaction { append EventActivityScheduled; write outbox item; update state -> WAITING_ACTIVITY }.

The discipline. Every external effect is mediated by an outbox/activity record keyed by operation_id. Direct effect-then-state (or state-then-effect) without the boundary is a violation.

See also: 46, 65

69. Time is a durable event, not a sleep — timers are persisted, not implied by wall-clock waits

HIGH SEVERITY.

The trap. A sleep feels like a timer — but it evaporates on restart and is invisible to replay and diagnosis. Its familiar disguise: *"time.Sleep(30s); state = TIMED_OUT"*. Timers must be persisted. time.Sleep is an implementation detail, never the source of truth: EventTimerScheduled(due_at) -> EventTimerFired(timer_id, observed_generation) -> transition WAITING -> TIMED_OUT. A restart must not lose or double-fire the timer.

The discipline. Deadlines/timeouts are recorded as durable timer events with due_at, not driven by an in-process sleep whose expiry sets state.

See also: 27, 50

70. Compensation is forward motion — rollback is fantasy once a side effect escaped

HIGH SEVERITY.

The trap. An undo feels available — but once an external effect happened, only compensation (a new forward action) can correct it. Its familiar disguise: *"delete local state and pretend the install never happened"*. Once a side effect has happened, rollback is fantasy unless the external world supports true undo; most systems need compensation — a forward recovery sequence: INSTALL_APPLIED -> VERIFY_FAILED -> ROLLBACK_REQUIRED -> ROLLBACK_STARTED -> ROLLBACK_COMPLETED, each a recorded state, not a silent local delete.

The discipline. Recovery from a committed side effect is modeled as explicit compensation states with their own events — never by deleting local state to feign that the effect never occurred.

See also: 48, 62

71. Retry is modeled state — attempt, delay, reason, and idempotency, not a hidden code loop

HIGH SEVERITY.

The trap. A for-loop feels like a retry — but it hides attempt count, backoff, reason, and idempotency from state and replay. Its familiar disguise: *"for i := 0; i < 5; i++ { doThing() }"*. Retries need state, count, delay, reason, and idempotency: ATTEMPT_FAILED -> RETRY_SCHEDULED(attempt=2, due_at=T) -> RETRYING -> ATTEMPT_SUCCEEDED. A retry buried in an in-process loop is invisible to diagnosis and unbounded to the graph.

The discipline. Retry is expressed as declared states/events with an attempt counter and a bounded schedule, not an opaque loop.

See also: 43, 69

72. Unknown is safer than invented — a workflow that can't prove truth enters UNKNOWN, it does not guess

CRITICAL SEVERITY.

The trap. Filling a gap with an assumption feels decisive — a wrong guess about current truth triggers destructive action. Its familiar disguise: *"missing record -> assume not installed"*. If the workflow cannot

prove current truth, it enters UNKNOWN or RECONCILE_REQUIRED, never a guessed concrete state: missing record -> UNKNOWN_NEEDS_OWNER_RECONCILE, not “assume not installed”. This is the workflow form of “absence of desired state is not permission to destroy”.

The discipline. Unprovable state resolves to an explicit UNKNOWN/reconcile state that requests fresh owner truth — it must not default to a concrete state that permits destructive next actions.

See also: 9

73. Projections must never drive progression — UI/queue/CLI/log views observe, they do not advance the workflow

HIGH SEVERITY.

The trap. A view reflecting absence feels like completion — but a projection derived from state must not become the cause of a state change. Its familiar disguise: *“task no longer in queue -> mark complete”*. UI, dashboards, queue views, task lists, CLI output, cached summaries, and logs are projections. They must not advance the workflow: a task-completion EVENT updates the queue projection — the queue projection going empty must not mark the task complete. Progression flows from events to projections, never backward.

The discipline. No workflow transition is triggered by the state of a projection. Projections are read-only derivations of authoritative state.

See also: 1, 122

74. A quorum/ownership-changing operation must be safe if the joining or leaving member fails halfway through

CRITICAL SEVERITY.

The trap. Adding capacity feels safe — but raising the quorum denominator before the member is real can strand the survivors below quorum, unable even to undo the change. Its familiar disguise: *“Add the new member as a full voter/owner now; if it dies we’ll clean it up”*. Any operation that changes quorum, voter count, replication eligibility, token ownership, or erasure-set survivability must remain safe if the joining or leaving member fails at the worst moment — mid-operation.

The discipline. Membership growth on a quorum/ownership substrate must enter through the least-dangerous role and be crash-safe at every intermediate step. Rollback of a not-yet-promoted member must not depend on a quorum the half-finished operation may have broken. This is the substrate-membership specialization of meta.circular_dependency_must_have_break_glass and meta.state_mutations_must_be_durably_committed_before_side_effects.

See also: 20, 41, 75, 79

75. A member must not be promoted into a topology-critical role until it has proven substrate-specific readiness

CRITICAL SEVERITY.

The trap. Admission and a successful start feel like readiness — but being allowed in is not the same as being caught up enough to carry quorum or ownership. Its familiar disguise: *“It was admitted / configured / reachable once / started, so promote it to voter/owner”*. Promotion to a quorum- or ownership-critical role

(voter, token owner, active erasure member) must be gated on readiness proof from the substrate itself, not on the facts that a member was admitted, configured, reachable once, or started as a process.

The discipline. The controller must obtain a substrate-native readiness signal before promotion and must retry rather than force-promote when the signal says "not yet". Promotion must never be driven by admission, config presence, or a one-time reachability probe.

See also: 14, 20, 74, 128

76. Distributed-data recovery must use the substrate's own recovery model, not a generic restart/delete/reinstall

CRITICAL SEVERITY.

The trap. Stop-delete-reinstall works for stateless services, so it feels universal — but for a replicated store it can destroy the only surviving replica or the ring's memory of ownership. Its familiar disguise: *"It's broken — stop the service, delete its data, reinstall/rejoin"*. Recovery of a distributed data substrate must use the recovery model that substrate defines, not a one-size stop/delete/reinstall pattern that is only correct for stateless services.

The discipline. Remediation/repair paths for a data substrate must dispatch the substrate's native recovery operation, gated on its topology state. Generic service restart/reinstall must not be applied to quorum/ownership/durability failures. Narrower, evidence-gated recovery mode still applies (meta.recovery_mode_must_be_explicit_and_narrower).

See also: 18, 58, 74

77. Before automating membership changes for a data substrate, its allowed/forbidden/transitional topology states must be defined — otherwise refuse destructive or quorum-changing actions

CRITICAL SEVERITY.

The trap. Automation feels like progress — but automating membership changes without a declared topology policy automates the incidents too. Its familiar disguise: *"Automate join/remove/promote now; we'll figure out the safe-state rules as incidents happen"*.

The discipline. If no topology policy exists for a substrate, automation must REFUSE destructive or quorum-changing actions and escalate. Automated membership drivers must encode the allowed/forbidden/transitional state set and their promotion/rollback/timeout gates explicitly. Related to meta.partition_response_must_be_predeclared and meta.topology_change_is_a_first_class_event.

See also: 41, 74, 80, 81

> *Field note — The system did not hang because the work was hard. It hung because someone wrote a promise and appointed no one to keep it.*

IV. On Dependencies, Topology, and Retreat

A general studies the road home before he marches. So must the architect study the recovery path before the failure — for a dependency you did not need on the happy path will be the one that kills you on the road back. Decide the answer to the partition before the partition arrives; to choose in the moment the network splits is not strategy, it is the bug arriving on schedule.

78. The critical path must not depend on non-critical services — a dependency you don't need will kill you on the recovery path

CRITICAL SEVERITY.

The trap. Convenience over resilience — adding a repository call to heartbeat feels useful, but when the repository is down the heartbeat dies too. Its familiar disguise: *"One more RPC won't hurt"*. A critical function (heartbeat, leader liveness, connection establishment) must not pick up a dependency on a non-critical service (repository, event bus, analytics, monitoring). When the non-critical thing fails, it takes down the critical path — not by crashing, but by blocking, flooding, or silencing.

The discipline. Heartbeat, leader liveness, and connection establishment paths must have zero dependencies on non-infrastructure services. Any RPC in a critical path must have a bounded timeout and must not block the caller on failure.

79. Every circular dependency must have a break-glass path that doesn't go through the cycle — a system that deploys itself must have a path that doesn't go through itself

CRITICAL SEVERITY.

The trap. Trust in the normal path — when repository is broken, deploying a fix through the repository feels like the right path, but the cycle is stuck. Its familiar disguise: *"The pipeline will fix itself"*. When A depends on B and B depends on A, the normal operational path forms a cycle. When either component breaks, the normal path through the other can't fix it — the cycle is stuck. This is the chicken-egg problem: the deploy pipeline can't deploy a fix for itself, the controller can't dispatch a fix for the controller, the repository can't publish a fix to the repository.

The discipline. Every self-referential deploy path (service that publishes/installs itself) must have a documented break-glass recovery procedure. The bridge pattern (scp verified binary from healthy peer, manual install, return to normal pipeline) is the canonical break-glass. Hot-deploying locally-built binaries is NOT a valid break-glass — no ldflags means checksum mismatch means the cycle breaks again.

80. Cluster topology is a dynamic input — code that caches it at startup operates on a snapshot that becomes a lie within minutes

CRITICAL SEVERITY.

The trap. At any given second the cluster looks fixed — N nodes, M services, these routes, this leader. Code written against that snapshot looks correct in isolation. But topology changes during normal operation — leader elections, VIP migrations, joins, drains, cert rotations, service-config updates. Each is a routine event, not an

exception. Code that doesn't subscribe to topology changes caches a snapshot that becomes false within minutes — and then makes decisions on the false snapshot until something else fails loudly enough to force a reload. Its familiar disguise: *"The endpoints I cached at startup are still good"*. Distributed system topology — peer addresses, leader identity, VIP holder, service ports, membership, certificate validity, route reachability — changes during NORMAL cluster operation. Tanenbaum's sixth fallacy ("the topology doesn't change") is false in every operator-grade system. Code that treats topology as a startup-time fact will operate on stale data within minutes of any normal cluster activity.

The discipline. All inter-service addresses, leader identities, VIP holders, service ports, node memberships, certificate validity, route reachability MUST come from a live source: - etcd watches on the canonical key - cluster controller RPC with bounded freshness - service-mesh discovery (xDS) - event-bus subscriptions for cluster events

See also: 3, 7, 24, 45

81. An actor's response to network partition must be decided BEFORE the partition occurs — choosing in the moment is the bug

CRITICAL SEVERITY.

The trap. A network partition is rare, so it feels reasonable to defer thinking about partition response until it happens. But partition response cannot be a runtime choice — different code paths under partition will choose differently, and the actor's behavior under partition becomes a function of which call site fired first. The CAP theorem isn't a permission to choose under fire; it's a forcing function to commit to a posture before the partition occurs so the system has consistent behavior under stress. Its familiar disguise: *"If we get partitioned, we'll figure out what to do"*. Brewer's CAP theorem says that during a network partition, a system can serve consistent reads (CP) OR remain available (AP), not both. The principle here is the operationalization: every actor or service in the system MUST predeclare its partition response — what it will do when it can't reach its quorum, its coordinator, its peers, or its sources of authoritative state.

The discipline. Every actor (service, daemon, workflow handler) must have a DOCUMENTED partition posture, named in either: - Its top-of-file @awareness annotation - Its README or doc/services/.md - The actor's gRPC service-level Doc string

See also: 7, 23, 27, 79

82. State that can move between nodes recovers via rebind; state that only replicates recovers via reinstall — rebind is much cheaper

CRITICAL SEVERITY.

The trap. Replication feels like the strong primitive — if your data is replicated, you can lose any single replica. But replication only handles DATA loss, not COMPUTE loss. When a node dies, the data is fine (replicated to other nodes) but the SERVICE PROCESS that was using it is gone. Recovery now requires re-establishing the service somewhere — which means full install, verify, start. That re-establishment IS the expensive part; replication did not help. Its familiar disguise: *"We have replication so we are covered against node loss"*. Eden (Black 1985 §3.2.3) made object mobility a first-class primitive. An Eden object could move between nodes for load balancing, and the same primitive handled recovery — "Should the node break, the object will be unavailable until it is repaired. Now suppose that we try to increase the availability of the object

by replicating its checkpoint file on two additional nodes… When the node on which it is running breaks, our object can be reactivated on another node, using the remotely checkpointed state."

The discipline. For new stateful service designs:

- Identify the service's PERSISTENT state (lives in Scylla, MinIO, etcd) versus its PROCESS state (lives in the running binary's memory).
- The process state should be ephemeral and reconstructible from persistent state on restart.
- The service binary should be available on every node that could host it; the release pipeline ensures this via desired-state convergence.
- The mobility primitive is "detach the running incarnation, reconnect to the same persistent state from a different node, resume."

See also: 3, 78, 80, 81

83. Replication preserves harvest under fault and reduces yield; partitioning preserves yield and reduces harvest — the choice maps to which dimension the workload tolerates losing

CRITICAL SEVERITY.

The trap. Replication has the cultural reputation of "more available" because every replica is a complete copy of the data, so the data survives any single failure. But that reasoning conflates DATA durability with QUERY availability. Replicated systems lose capacity under fault (some queries get refused); partitioned systems lose data per answer (every query returns, with a fragment). Neither is inherently better; they preserve different things. The choice has been silently embedded in every persistent-store decision in the system. Its familiar disguise: *"Replicated is more available than partitioned"*. Brewer's quantitative analysis: a 2-node cluster under one node loss either drops to 50 percent yield (replicated version) OR drops to 50 percent harvest (partitioned version). The DQ value — data per query times queries per second — drops by half in both cases. Replicas preserve D and reduce Q; partitions preserve Q and reduce D.

The discipline. For every new persistent-state subsystem, the design document must state:

- WHETHER the data is replicated, partitioned, or hybrid.
- WHICH availability dimension this preserves under single-node fault (yield, harvest, or both within a tolerance).
- WHICH availability dimension is reduced when the tolerance is exceeded.
- WHETHER the workload tolerates the chosen reduction — explicit reasoning, not "it should be fine."

See also: 3, 7, 34, 81

84. Replication does not preserve yield unless surviving nodes have spare capacity to absorb redirected load — and that capacity must be measured, not assumed

CRITICAL SEVERITY.

The trap. Replication feels like a complete answer to node loss — the data is on other nodes, so requests can be served from those. But "can be served" assumes the surviving nodes have the spare capacity to absorb the failed node's load. At typical production utilization (80 percent reported by Brewer), surviving nodes are nearly saturated even at full strength; absorbing the load of a failed peer can push them over the edge. The cascade is invisible until it happens. Its familiar disguise: *"We replicate so we can handle node loss"*. Brewer named this

the LOAD REDIRECTION PROBLEM. “The traditional view of replication silently assumes that there is enough excess capacity to prevent faults from affecting yield… Under high utilization, this is unrealistic.” His arithmetic shows the cost is sharper than intuition suggests.

The discipline. For every cluster-scope subsystem that uses replication for availability:

- NAME the maximum k (concurrent failures) the subsystem must absorb without yield reduction.
- MEASURE current utilization at full strength.
- COMPUTE the overload factor — utilization times n/(n-k) — and verify the product is below the subsystem’s saturation threshold.
- If the product exceeds threshold, either add nodes (reduce per-node utilization) OR design graceful degradation (reduce DQ at the application layer when survivor saturation is detected).

See also: 43, 52, 78, 83

> *Field note — You do not discover your circular dependency in the design review. You discover it at 3 a.m., when the thing that would fix it is the thing that is down.*

V. On the Operator’s Eye

The screen is the only ground the operator can see, and so it is where the war is truly won or lost. A green badge that shines because the model in memory said so is a sentry reporting a peace that does not exist. Certainty is part of the value: loading, stale, unknown, optimistic, and confirmed must each look like exactly what they are.

85. Every screen claim binds to the correct authority — desired, cached, optimistic, and confirmed state must not collapse into one visual meaning

CRITICAL SEVERITY.

The trap. Render whatever state is at hand — it is available, typed correctly, and probably right. Its familiar disguise: *“The badge is green because the model in memory said so”*. The screen is a projection of system truth, and a projection inherits the authority of its source. A claim about runtime health must trace to the runtime authority; a claim about installation to the installed-state record; a claim about permission to RBAC; a claim about action outcome to workflow receipts. Desired state, cached state, optimistic local state, and runtime-confirmed state are four different epistemic levels — rendering any of them through the same visual meaning makes the screen lie with confidence. This is the UI projection of meta.storage_is_not_semantic_authority: a component store or DOM node holding a value does not make that value true.

The discipline. For every status badge, indicator, count, or claim rendered: name the authority that backs it. If the binding cannot be named, the element is displaying guesswork and must be changed to either bind correctly or visibly mark its uncertainty.

See also: 1

86. Certainty is part of the value — loading, stale, unknown, optimistic, and confirmed must each look like what they are

CRITICAL SEVERITY.

The trap. Reduce visual noise by collapsing in-between states into the nearest clean state. Its familiar disguise: *“One spinner and one green check cover every situation”*. Nielsen’s first heuristic says the system must keep users informed of its status. The operational version is stronger: the CERTAINTY of a value is part of the value. Loading is not empty; stale is not current; unknown is not healthy; optimistic is not confirmed; denied is not zero results. Each epistemic state must be visually distinguishable, because an operator acts differently on “confirmed down” than on “status unknown”. This is the UI projection of meta.authority_must_express_uncertainty and meta.fallback_must_degrade_semantics: a screen that renders a fallback or cached value in the same visual form as confirmed truth has absorbed the uncertainty the operator needed.

The discipline. Before shipping any state-rendering component, enumerate which of empty/loading/unknown/stale/optimistic/denied/failed/confirmed it can occupy, and verify each occupied state has a distinct, non-confusable rendering. A state the component can occupy but cannot display is a lie waiting for traffic.

See also: 21, 22

87. One meaning, one visual language — the same operational state must render identically everywhere it appears

HIGH SEVERITY.

The trap. Style each screen locally — consistency is a polish concern for later. Its familiar disguise: *"Red means failed on this screen, disabled on that one, and destructive on a third"*. Operators build a visual vocabulary: this shade means degraded, this icon means converged, this badge means permission denied. Every screen that breaks the vocabulary forces re-learning and invites misclassification under stress. This is the UI projection of meta.identity_computation_must_be_invariant: one state, one meaning, one canonical rendering, everywhere. The vocabulary belongs to the design system layer (component library, tokens), and screens must consume it rather than re-invent it.

The discipline. Before introducing a new color/icon/badge for an operational state, check whether the state already has a rendering in the shared vocabulary; reuse it. Before reusing an existing color/icon for a NEW meaning, verify it does not already carry a different operational meaning elsewhere.

See also: 2

88. An enabled destructive button is an assertion — permission and risk state must be confirmed before the action is offered

CRITICAL SEVERITY.

The trap. The backend enforces RBAC anyway, so the frontend gate is redundant work. Its familiar disguise: *"Enable the button and let the backend reject it"*. An enabled action is itself a screen claim: "you may do this, and it is sane to do now". Offering a destructive action (wipe, topology apply, node remove, force rollback) without confirmed RBAC permission and visible risk state asserts safety the system has not granted. Backend enforcement protects the cluster but not the operator's mental model — a button that fails on click after looking available has already lied. Error prevention beats error handling: gate the offer, not just the execution, and make duplicate submission impossible rather than recoverable.

The discipline. Before rendering any mutating or destructive control: bind enablement to an RBAC validation result (not assumption), surface the risk state the action carries, expose the reason when disabled, and guard submission with idempotency or single-flight.

89. Failure is when the screen matters most — errors must preserve context and offer a path, never blank the operator's world

HIGH SEVERITY.

The trap. Replace the broken region with a clean generic error so the UI looks controlled. Its familiar disguise: *"Something went wrong. [Reload]"*. The moment something fails is the moment the operator needs the screen the most. An error path that erases selection, hides which component failed, discards last-known data and its timestamp, or collapses a specific backend failure into a generic message converts a recoverable incident into a blind one. Recovery requires context: what failed, what was known last and when, what is still trustworthy on screen, and what safe action exists. Full-screen blanking for a partial failure is the UI equivalent of a process crash taking down healthy siblings.

The discipline. Generic catch-all error surfaces are acceptable only as the outermost last resort, never as the first response to a component failure.

90. Operational claims carry their provenance — source, node, generation, and version are shown, not remembered

HIGH SEVERITY.

The trap. Strip identifying detail to keep the layout clean — power users know the context. Its familiar disguise: *"The operator can check which node that was on the other screen"*. Recognition-over-recall in an operator UI means a claim's provenance travels with the claim: which node reported it, which generation it belongs to, which version produced it, when it was observed. Forcing operators to reconstruct provenance from memory or other screens strips the scope off an assertion — the perception-layer twin of meta.assertions_must_carry_their_scope. "Healthy" without which-node and as-of-when is not information; it is reassurance.

The discipline. When rendering aggregated or summarized state, preserve the scope: counts say out-of-how-many, statuses say as-of-when, identities say which-node/which-version. If the layout cannot fit the provenance, make it one interaction away — never absent.

See also: 25

91. Meaning must survive presentation loss — color removal, layout collapse, keyboard-only, and theme changes must not change what the screen asserts

CRITICAL SEVERITY.

The trap. Treat accessibility and responsive layout as cosmetic passes after the screen works. Its familiar disguise: *"The warning is the red tint on the row"*. WCAG's four principles, re-grounded for operators: any single presentation channel can be lost — color (vision deficit, monochrome, glare), layout (mobile collapse, overflow, truncation), pointer (keyboard-only), styling (theme/skin change) — and the screen's operational assertions must survive each loss. A destructive-action warning that exists only as a hover, a color, or a desktop-width element is a warning that conditionally exists. Skins may change presentation; skins must not change meaning.

The discipline. For every critical state rendering, verify the meaning is carried by text or structure, not only by color/hover/animation/position; that responsive collapse never drops warnings or risk indicators; and that theme changes cannot reduce a critical signal below visibility.

92. Decoration must not impersonate authority — cosmetic elements and generated prose stay visually subordinate to authoritative state

HIGH SEVERITY.

The trap. Make it feel alive — placeholder data, optimistic animations, and friendly AI prose improve the experience. Its familiar disguise: *"An illustrative all-green dashboard hero while two nodes are degraded below the fold"*. Visual hierarchy is an authority claim: whatever dominates the screen is what the operator believes first. Decorative elements that LOOK like state — sample charts, placeholder counts, optimistic success animations, AI-generated prose with status-shaped sentences — compete with real authority and sometimes win. Material's hierarchy doctrine, re-grounded: important operational truth must dominate

cosmetic layout, and nothing cosmetic may be confusable with truth. AI may decorate the explanation; AI must not invent the state.

The discipline. Before adding any decorative, placeholder, illustrative, or generated element: verify it cannot be read as live state (no status colors, no realistic counts, no checkmarks), and verify the screen's visual weight ranks authoritative state above decoration — risk and failure must never sit below the fold of an ornament.

93. Simplicity must not hide operational truth — a clean lie is worse than a messy truth

CRITICAL SEVERITY.

The trap. Worship clean — hide the ugly details because most users will not need them. Its familiar disguise: *"We removed the node column and the generation badge — the dashboard looks much cleaner now"*. A UI may simplify presentation, but it must not hide operational facts required for a safe decision. Node identity, authority source, generation, freshness, failure state, partial quorum, missing receipts, and runtime evidence must remain visible whenever they affect the meaning of the screen. Modern design culture rewards "clean"; a cluster console is cockpit instrumentation — the gremlins (stale generation, node mismatch, failed receipt, wrong authority) are not clutter, they ARE the information. GOV.UK's principle done right: do the hard work to make it simple — simplify the path to truth, never the truth itself.

The discipline. Before removing, collapsing, or hiding any operational detail in the name of visual simplicity, ask: can a safe decision still be made without it on THIS screen? If the detail affects meaning, it may move one interaction away at most — never disappear. Declutter decoration, never evidence.

See also: 4, 90

94. The operator remains in control — automation, auto-refresh, optimistic updates, and AI assistance must not change state without visible intent and consent

CRITICAL SEVERITY.

The trap. Be helpful — pre-select, auto-apply, auto-refresh, and auto-fix so the operator has less to do. Its familiar disguise: *"The wizard auto-applied the recommended topology while the operator was reading it"*. The UI must not let automation change operational state without clear operator intent, visible authority, and a recoverable outcome. This covers auto-refresh that discards context, wizards that pre-execute, optimistic updates that render unconfirmed mutations as done, and AI suggestions that shade into AI actions. The aviation lesson (automation surprise, mode confusion) applies verbatim: the operator must always be able to answer "what is it doing, why did it do that, what will it do next". Every automated state change must be attributable on screen — what acted, under what authority, with what evidence.

The discipline. Before adding any automatic behavior that mutates state or replaces operator decisions: require explicit consent at the point of action, render automated changes as attributed events (actor, authority, time), and keep a visible path to halt or reverse the automation. Suggestion and execution must be visually and mechanically distinct.

See also: 46, 88

95. Dangerous actions are reversible or guarded — undo, dry-run, preflight, or explicit risk display before execution, with no fast path around the guard

CRITICAL SEVERITY.

The trap. Confirmation dialogs annoy experts — add a shortcut that bypasses the ceremony. Its familiar disguise: *"Power users can skip the confirmation with shift-click"*. Any destructive, security-sensitive, or topology-changing action must provide at least one of: an undo path, a dry-run/preview, a backend-backed preflight, or an explicit risk display with confirmation. Shneiderman's reversal rule, hardened: where true undo is impossible (wipe, key rotation, node removal), the guard before execution carries the full weight. And his shortcuts rule, bounded: expert speed may skip ceremony, never safety — a keyboard shortcut or bulk action that bypasses RBAC checks, confirmation, preview, or risk display is a privilege escalation through the UI layer.

The discipline. For every irreversible action: name its guard (undo | dry-run | preflight | risk-confirm). Verify every path to the action — button, shortcut, bulk operation, API-driven UI action — passes through the same guard. A guard with a bypass is not a guard.

See also: 88

96. Every workflow yields closure — the final state, receipt, or required next action must be visible; a disappeared spinner is not a result

HIGH SEVERITY.

The trap. End the interaction at submission — the backend will take it from here. Its familiar disguise: *"The dialog closed and the spinner went away, so it probably worked"*. Every operational workflow must end with explicit closure: terminal state, receipt identity, error with diagnostic context, or the next required action. A submitted form, dismissed dialog, or vanished spinner proves only that the UI moved on — not that the operation completed. This is the perception twin of meta.write_creates_completion_obligation and meta.half_done_must_not_look_done: dispatch is not completion, and the screen must not let an intermediate state read as a terminal one. Progress indication must report real work — a progress element not bound to backend workflow evidence (run id, step receipts, phase) is animation impersonating progress.

The discipline. Every mutating flow must render its terminal state from workflow authority (receipt, terminal status, error) — never infer it from dispatch success or UI transition. Progress bars and spinners must bind to backend progress evidence or visually declare themselves indeterminate waits.

See also: 40, 41

97. AI assistance is explainable and bounded — summaries cite source, time, and confidence; recommendations show the evidence boundary

CRITICAL SEVERITY.

The trap. Let the assistant speak naturally — citations and uncertainty markers make the text clunky. Its familiar disguise: *"Everything looks healthy! — generated from a model with no cluster evidence in context"*. AI-generated UI text may summarize or decorate authoritative evidence, but must not invent state, collapse uncertainty, omit source authority, or recommend actions without showing the evidence boundary — what the AI actually saw, from where, as of when, and how confident it is. An unsourced fluent paragraph about cluster state is the most dangerous element on a screen: it carries the authority of prose with the provenance of nothing. Where evidence is partial, the text must say so; where the AI is uncertain, the rendering must carry the uncertainty rather than absorb it.

The discipline. Every AI-generated claim about system state must be traceable on screen to its evidence: source authority, observation time, and confidence or evidence-completeness. Recommendations must

distinguish “the evidence shows” from “the model suggests”. Unsourced generated text may exist only in clearly decorative, non-status form.

See also: 22, 92

98. The screen serves the operator’s task, not the system’s schema — API shape is not information architecture

HIGH SEVERITY.

The trap. Mirror the backend API in the UI — it is complete, already typed, and easy to generate. Its familiar disguise: *“One page per proto message, one form field per proto field, one table per RPC”*. Operators arrive with goals — “is the cluster safe after the power cut”, “why is this release stuck”, “take this node out safely” — not with the desire to browse resources. A UI organized by backend schema (one screen per service, one widget per RPC) forces the operator to perform the join across screens in their head, under stress. This is the default failure of GENERATED UIs: an AI agent given a proto will instinctively render the proto. The task, not the type system, owns the information architecture; the schema is plumbing below it.

The discipline. Before creating a screen, name the operator task it serves and the decision it enables. If the screen’s structure mirrors a proto/RPC rather than a task, justify it (raw resource inspectors are legitimate — as explicitly secondary surfaces). Group what the task needs together, even when it crosses services.

99. Notification volume must match operator capacity — an alarm flood is a perception outage

HIGH SEVERITY.

The trap. More alerts means more safety — surface everything and let the operator filter. Its familiar disguise: *“Every finding, state change, and retry raises a toast — the operator will appreciate the transparency”*. Every alert the UI raises must be rationalized — actionable, prioritized, deduplicated, and rate-aware. A flood of toasts during a cascade buries the one warning that matters under twenty that don’t, exactly as the doctor event-amplification incident flooded the event bus on the backend. Standing alarms (permanently red badges nobody acts on) are the static version of the same failure: they train the eye to ignore red.

The discipline. Every notification class must declare: what action the operator should take, its priority, and its dedup/aggregation rule. Cascades must collapse into one root-cause notification with detail beneath, not N symptom toasts. Audit standing indicators: any permanently-asserted warning that has no pending action is alarm pollution to fix at the source or re-classify.

See also: 43, 44

100. Elements that carry operational meaning must be individually addressable — a styled span with no identity is invisible to tooling, tests, and the operator’s tools

HIGH SEVERITY.

The trap. It renders correctly, so it’s done — identity is for forms and inputs, not display elements. Its familiar disguise: *“Status counts and filter pills rendered as anonymous with inline styles — no id, no data-bind, no data-”*. *Every UI element that carries operational meaning — a status count, a health badge, a filter state, a node identifier — must be individually addressable: an id, a data-bind, or a data-* attribute. Anonymous styled spans are invisible to the things that make an operator UI trustworthy: automated tests can’t assert on them,

accessibility tooling can't name them, screen-scraping diagnostics can't read them, and a targeted refresh can't update them without re-rendering the whole region (which destroys operator context). Addressability is the UI projection of meta.code.extension_points_must_be_explicit and the precondition for meta.ui.state_certainty (you cannot mark one datum stale if you cannot name it). Discovered on the admin incidents page: filter pills carried data-filter, but the count spans inside them carried nothing.

The discipline. Before shipping a component that renders operational state, verify each data-carrying or interactive element has a stable identity (id / data-bind / data-*) — not just the container. Counts, badges, and status text that sit inside an identified parent still need their own handle if anything (test, refresh, a11y, diagnostic) must address them individually.

See also: 86, 90, 118

101. The governing contract must be made explicit before a change can be called resolved

CRITICAL SEVERITY.

The trap. A green test feels like proof of resolution — but it only proves the change matched a hidden oracle, not that the change respects the contract the code is actually bound by. Its familiar disguise: *"The tests pass, so it's fixed."*. Before repairing anything, AWG's first duty is to identify, infer, or propose the governing contract/invariant for the code under change. You cannot ask "was this respected?" until a contract exists to be respected.

See also: 6, 35, 103

102. A repair contract must define not only the intended rule, but also the scope in which the rule may be applied

HIGH SEVERITY.

The trap. Once an agent sees a valid contract, it can generalize into neighboring files or behavior paths unless someone explicitly forbids it. Its familiar disguise: *"The contract is true, so any semantically related edit is fair game."*. A repair contract must define not only the intended rule, but also the scope in which the rule may be applied. Before an issue can be considered contract-resolved, the governing contract must identify the required scope, allowed related scope, out-of-scope areas, required behavior paths, and forbidden broadening.

See also: 25, 101, 103

103. No resolution without a respected contract — green tests are not enough

CRITICAL SEVERITY.

The trap. Treating hidden-test pass as the definition of success rewards guessing the oracle and hides the dangerous case: a patch that passes the covered tests while violating an uncovered contract. Its familiar disguise: *"It passes the benchmark, ship it."*. The operational form of meta.contract_must_be_explicit_before_resolution. A change may be labelled "resolved" only when ALL hold: 1. a contract is explicit (identified or made explicit), and 2. the patch respects that contract (gated, not assumed), and 3. the respect is supported by evidence (the contract's detect rule, a red->green test, or a cited grounding). A patch that passes tests but has no identified contract is an oracle match, not an AWG-valid resolution. A patch that respects the contract but fails the hidden tests may still be architecturally honest. These are distinct outcomes and must be reported separately, never averaged.

See also: 6, 101

> *Field note — The operator did not misread the screen. The screen misrepresented the system, in good faith, in a pleasant shade of green.*

VI. On Arrangement, Weight, and Visual Command

Spacing is information; equal spacing makes unrelated facts look like kin. The layout is an argument the eye believes before the mind has read a word — so let weight, order, and grouping follow the operator's decision, never the designer's mood. Safety evidence outranks decoration. Always.

104. Visual weight, order, spacing, and grouping must match the operator's decision path — safety evidence outranks decoration, always

CRITICAL SEVERITY.

The trap. Lead with the reassuring summary and tuck the diagnostics below the fold — that's what dashboards look like. Its familiar disguise: *"Big hero card: 'ObjectStore looks great!' — tiny footer: applied_generation != desired_generation"*. The visual weight, order, spacing, and grouping of a screen must match the operator's decision path. Information required to judge safety, authority, freshness, risk, and outcome must be more prominent than decorative summaries, secondary metadata, or convenience actions. Whatever dominates the screen is what the operator believes first and acts on fastest — a layout is itself a priority claim. This is the composition anchor: the perception twin (decoration_must_not_impersonate_authority) bans decoration LOOKING like truth; this principle bans truth being PLACED like decoration.

The discipline. For every screen, write the decision path first (what must the operator judge, in what order) and verify the layout's prominence order matches it. Drift, risk, and required actions never render visually weaker than summaries or branding. The good shape: top banner "Topology drift detected"; main panel desired vs applied; action locked until risk evidence is reviewed.

See also: 92, 93

105. Visually grouped elements must share real semantic relationship — layout must not tell a story the data does not

HIGH SEVERITY.

The trap. Group whatever fits nicely in the card — the grid looks balanced that way. Its familiar disguise: *"desired_generation and runtime_healthy share one green card — the operator reads one confirmed truth"*. Humans group visual elements before they read them (Gestalt: proximity, similarity, common region, closure). Elements grouped by border, background, alignment, or spacing are perceived as one meaning — so a card that mixes authority sources (a desired-state value beside a runtime value), states, actions, or risks asserts a relationship the data does not have. The UI must not accidentally tell a story through layout: grouping IS a claim, and claims bind to authority (meta.ui.screen_claim_must_bind_to_authority).

The discipline. For every visual group (card, panel, row, bordered region), name the semantic relationship its members share — same authority, same subject, same action family. Mixed-authority groups must visibly separate their members (distinct sub-regions, per-claim provenance), never blend them under one status color.

See also: 85, 86

106. Spacing is information — equal spacing makes unrelated facts look related, and grouping gaps must mean grouping

HIGH SEVERITY.

The trap. Apply one uniform spacing scale everywhere for visual rhythm. Its familiar disguise: *"A uniform 16px grid over the whole page — node identity, runtime state, and marketing copy all equidistant"*. Proximity is the strongest Gestalt grouping force: things closer together are read as more related. Spacing is therefore semantic bandwidth, not aesthetic rhythm — uniform spacing flattens real relationships (which evidence supports which claim, which action belongs to which resource) and invents false ones. Related evidence-claim pairs sit closer than unrelated neighbors; boundaries between authority domains carry wider gaps than boundaries within one.

The discipline. When laying out evidence, claims, and actions: spacing within a semantic unit is tighter than spacing between units, consistently. Before equalizing spacing for visual neatness, check what relationship the equalization erases or fabricates.

See also: 105

107. Screen area and interaction prominence must reflect operational importance — critical evidence is never smaller than decoration

HIGH SEVERITY.

The trap. Size elements by how often they're used, or by what makes the layout feel dynamic. Its familiar disguise: *"The destructive Apply button is the biggest, brightest element; the wipe-risk note is 11px gray"*. Screen area, visual weight, and interaction prominence must reflect operational importance. Critical evidence, risk, authority, and required operator decisions must not be visually smaller, weaker, or harder to reach than low-risk decoration or convenience content. A destructive action rendered with more prominence than its own risk evidence inverts the operator's reading order — they see the button before the reason not to press it.

The discipline. Rank the screen's elements by operational weight (risk > authority evidence > status > navigation > decoration) and verify the rendered size/contrast/position order does not invert it. An action's prominence must never exceed the prominence of the evidence required to judge it.

See also: 88, 104

108. Status colors have stable contracts — success means confirmed runtime truth only, and no color serves two conflicting roles

HIGH SEVERITY.

The trap. Pick colors per screen for visual appeal; the palette is a styling concern. Its familiar disguise: *"Green for the brand accent, green for selected rows, green for healthy — three meanings, one hue"*. Theme and status colors must hold stable meanings across the entire interface. The Globular palette is role-based: brand, surface, info, success, warning, danger, unknown, stale. The two load-bearing rules: success color marks CONFIRMED RUNTIME TRUTH ONLY — never desired state, never optimistic updates, never decoration; and unknown/stale are first-class roles, because a palette without them is exactly how a fallback value ends up rendered green. A color reused for a conflicting semantic role (success + selection, danger + emphasis) poisons the operator's learned vocabulary.

The discipline. Every color use maps to one palette role; every role maps to one epistemic meaning. Before using success color, name the runtime authority confirming the value. Before reusing any status hue for a non-status purpose, pick a different hue.

See also: 86, 87, 91

109. Type expresses the information hierarchy — title, status, evidence, warning, and metadata must be visually distinguishable at a glance

HIGH SEVERITY.

The trap. One type style keeps the UI minimal and consistent. Its familiar disguise: *"Everything is 14px medium gray — the wipe warning reads like a caption"*. Typography is the cheapest, most robust hierarchy channel the screen has — it survives color loss, theme changes, and small screens. The information classes on an operator screen (title, status, evidence values, warnings, metadata/provenance) must be typographically distinguishable: an operator scanning under stress must be able to find the warning and the evidence without reading everything. Flat typography forces serial reading; hierarchy enables triage.

The discipline. Define type roles alongside color roles (title, status, evidence, warning, metadata) and verify each information class renders in its role. Warnings and risk text are never typographically weaker than body text. Numeric evidence (generations, counts, hashes) renders in a style that survives skimming.

See also: 91, 104

110. Theme tokens encode semantic roles, not preferences — components consume success/warning/danger/stale/unknown, never raw values

HIGH SEVERITY.

The trap. Style components directly — tokens are ceremony for a one-app design system. Its familiar disguise: *"color: #4caf50 hardcoded in the status badge component"*. Theme colors, spacing, typography, and elevation must be defined as role-based tokens, and components must consume semantic tokens (success, warning, danger, stale, unknown, surface, emphasis…) — never one-off stylistic values. Tokens are the enforcement mechanism for the whole composition category: with role tokens, "success means confirmed runtime truth" is one definition; with raw values it is a thousand scattered decisions. Tokens are also what makes skin/theme changes safe — skins remap token values, they cannot remap meaning (meta.ui.meaning_must_survive_presentation_loss).

The discipline. No raw color/size/weight literals in components for anything carrying operational meaning — semantic token references only. New visual meaning requires a new token role, not a new hex value. This is the first composition principle to mechanize when frontend Tier-2 lands (a token-only lint is a trivial Semgrep/ESLint rule).

See also: 87, 108

> *Field note — The eye obeys the layout before the mind reads the label. Put the warning where the hand is already moving, or do not bother to write it.*

VII. On Boundaries, Reuse, and the Shape of Code

A module earns its life by hiding more complexity than its interface adds; the rest are names, files, and imports that hide nothing and give the bug a place to sleep. Contracts outlive the fashion of their implementation. Reuse follows meaning, never resemblance — two strangers welded back to back do not become one traveller.

111. A reusable unit is a stable semantic concept — explicit contract, hidden complexity, owned lifecycle, inspectable behavior, explicit extension

CRITICAL SEVERITY.

The trap. Extract whatever repeats into a shared unit and let consumers adapt to it. Its familiar disguise: *"A 'shared' component whose consumers import its internals, patch its DOM, and break on every refactor".* A reusable unit must represent a stable semantic concept with an explicit public contract, hidden internal complexity, owned lifecycle (subscriptions, timers, listeners, async work — acquired and released locally), inspectable runtime behavior, and extension points that do not require consumers to depend on private implementation details. Every other structure principle elaborates one clause of this one. done right is the shape: the tag is a contract, the internals are nobody's business, the lifecycle ends at disconnectedCallback.

The discipline. Before extracting or publishing a reusable unit, write its boundary first: name the concept, the inputs, the outputs/events, the states it exposes, the extension points, and what its lifecycle acquires and releases. If the boundary cannot be written without describing internals, the unit is not ready to be shared.

See also: 87

112. Contracts outlive implementation fashion — callers depend on semantic inputs/outputs/events, never private structure

CRITICAL SEVERITY.

The trap. Expose whatever is convenient now — internal structure, framework objects, generated classes; we can clean it up later. Its familiar disguise: *"The admin console imports a component's internal store because 'it was right there'"*. Reusable code must expose stable contracts that survive implementation changes, framework trends, and internal refactors. Callers depend on semantic inputs, outputs, events, and states — not private structure. Hyrum's Law is the enforcement pressure behind this: with enough consumers, EVERY observable behavior becomes a contract whether you meant it or not — so the observable surface must be deliberate and minimal, and anything observable-but-private must be made genuinely unobservable (Shadow DOM, unexported symbols, reserved fields) rather than merely documented as off-limits.

The discipline. Before changing a unit's public surface, enumerate what consumers can observe — not just what is documented. Before publishing one, minimize that observable surface. A consumer found depending on private structure is a boundary failure to fix in the unit, not just in the consumer.

See also: 2

113. Hide complexity, never truth — the boundary must expose the states, failures, side effects, and authority assumptions callers need

HIGH SEVERITY.

The trap. A clean API means fewer states and no error types — absorb the mess inside. Its familiar disguise: *"The component swallows the gRPC error and renders its cached value — 'the caller shouldn't have to care'"*. A component or module may hide internal complexity — that is its job — but its public boundary must honestly expose the states, failures, side effects, and authority assumptions callers need. Encapsulation must not become concealment of operational truth: a boundary that absorbs connection errors, collapses unknown into a default, or hides which authority its data came from has converted information hiding into lying (the structural root of meta.connection_errors_must_not_be_absorbed and meta.fallback_must_degrade_semantics). Hiding complexity is good. Hiding truth is poison.

The discipline. For every boundary: internals (algorithms, markup, storage layout, retries) are hidden; truth (error states, staleness, uncertainty, authority source, side effects) is exposed in the contract. If simplifying the API required deleting a state callers would act on differently, the simplification went too far.

See also: 21, 24, 85

114. Reuse follows meaning, not resemblance — shared code represents one coherent concept, never a visual or structural coincidence

HIGH SEVERITY.

The trap. Deduplicate anything that looks similar — DRY means zero repetition. Its familiar disguise: *"These two screens both have a table, so I made GenericMegaTableOfEverything with 14 boolean props"*. Code should be reused only when the reused unit represents one coherent concept or responsibility. Merging unrelated use cases because their current markup, fields, or control flow look similar creates a unit with as many reasons to change as it has consumers — the definition of low cohesion. Duplication is far cheaper than the wrong abstraction: two similar tables that serve different operator tasks will diverge, and the shared mega-component becomes a flag-riddled hostage. This is a top AI-generation failure: models pattern-match on surface resemblance, not semantic identity.

The discipline. Before extracting shared code, name the ONE concept the unit represents and verify every consumer means that concept — not merely renders the same shape today. When an existing shared unit grows mode flags per consumer, split it along the real concepts instead of adding the next flag.

See also: 98

115. Compose through standard protocols first — platform contracts (DOM, events, HTTP, gRPC, a11y) before private framework mechanisms

HIGH SEVERITY.

The trap. Use the framework's idiomatic mechanism for everything — that's what it's for. Its familiar disguise: *"Two components coordinate through a framework-specific global store nobody can observe from outside"*. Components should compose through platform or project-standard protocols before private framework mechanisms: browser contracts (custom elements, attributes/properties, DOM events, slots, CSS custom properties, accessibility semantics), network contracts (HTTP, gRPC), and typed project contracts — over hidden global state or framework-locked coupling. Standard protocols survive framework churn, are

observable with ordinary tools, and let units built years apart compose. This is where Web Components earn their place: the composition surface IS the platform.

The discipline. When wiring units together, reach for the standard protocol first and require justification to descend into a framework-private mechanism. Cross-unit coordination through module-level mutable state or framework context that crosses semantic boundaries is a coupling decision — make it explicitly or not at all.

See also: 112

116. Debuggability is correctness — structure must preserve inspection, tracing, and a path from runtime behavior back to source intent

HIGH SEVERITY.

The trap. Accept opaque magic for less boilerplate — productivity now, archaeology later. Its familiar disguise: *"Five build layers later, the rendered DOM has no relationship anyone can trace to the source tree"*. Code structure must preserve the ability to inspect, trace, and diagnose behavior with ordinary tools. Abstractions that hide state transitions, generated markup, errors, network calls, or ownership boundaries reduce system correctness even when they reduce boilerplate — operational code is read in anger at 3am, and "pretty productivity fog" is a real cost paid then. The same rule binds the build pipeline: transpilers, generators, and framework compilers must preserve a traceable path from runtime behavior back to source intent; a generated layer nobody can map back to source is too expensive for operational code regardless of what it saves at write time.

The discipline. For every abstraction or build step adopted, answer: at runtime, can a developer with browser devtools / delve / journalctl see the states, events, and calls this layer manages, and map what they see back to source? If the answer requires "install the framework's special devtools and hope", weigh that as real cost, not zero.

See also: 44

117. Local state caches and stages — it never becomes the authority for domain truth, permission, completion, or health

CRITICAL SEVERITY.

The trap. The component already has the value — asking the service again is wasteful. Its familiar disguise: *"The component 'knows' the workflow succeeded because its own submit handler set done=true"*. Component-local state may cache, stage, or render information, but must not become the authority for domain truth, permission, workflow completion, or runtime health. Authority-bearing state remains bound to its owning service or contract; the local copy is a mirror with a freshness obligation. This is the structural enforcement of meta.storage_is_not_semantic_authority at component scale — and the generative parent of ui.data_holder_is_cache_not_authority. The four questions for any field a unit holds: which layer owns this truth, which contract reads it, when does my copy expire, and what marks it stale.

The discipline. Classify every piece of unit-local state as either local UI state (selection, drafts, expansion) or a cache of authority-owned truth. Caches name their authority and their invalidation trigger. A unit answering an authority question (may the user do X? did Y complete?) from its own state without a freshness contract is a violation.

See also: 1, 85

118. Variation happens at declared extension points — slots, properties, events, adapters — never by reaching into private internals

HIGH SEVERITY.

The trap. Consumers can always override what they need — flexibility through openness. Its familiar disguise: *"The consumer customizes the component by querySelector-ing into its shadow root and patching styles"*. Reusable modules must define where variation is allowed: slots, properties, events, interfaces, configuration, adapters, callbacks. Consumers must not customize behavior by reaching into private internals, monkey-patching, or relying on undocumented structure — every such reach converts an internal into load-bearing API (Hyrum's Law, self-inflicted). The Web Components mapping makes the vocabulary concrete: attributes/properties are inputs, events are outputs, slots extend content, CSS custom properties/parts extend styling, imperative methods only where declarative contracts cannot express the need.

The discipline. When a consumer needs variation the unit does not offer, add the extension point to the unit — never patch around the boundary. A review that finds internals-reaching treats it as the unit's gap AND the consumer's violation, and fixes both.

See also: 112

119. Modules must be deep — an abstraction earns existence by hiding more complexity than its interface adds

HIGH SEVERITY.

The trap. More layers means more structure — wrap everything in a manager/service/util for tidiness. Its familiar disguise: *"Six 5-line wrapper helpers, each adding a name, a file, and an import — and hiding nothing"*. A module is worth its boundary only when the complexity it hides exceeds the complexity its interface adds. Shallow units — wrappers that rename, pass-through layers, single-caller helpers extracted for "cleanliness" — add interface area, indirection, files, and naming burden while hiding nothing; they make the system HARDER to understand while looking tidier. This is among the most common AI-generation failures: models produce layers because layers look like architecture. Few deep units beat many shallow ones, in Go packages and UI components alike.

The discipline. Before creating a wrapper, layer, helper, or base class, state what complexity it hides that callers would otherwise carry. "It groups related calls" or "it shortens a name" hides nothing. Prefer inlining a shallow abstraction over preserving it; prefer widening a deep unit over stacking a thin one on top.

See also: 114

120. A framework dependency must earn its rent — durable leverage greater than its coupling, lifecycle, build, debugging, and portability costs

WARNING SEVERITY.

The trap. Everyone uses framework X — starting without it is reinventing wheels. Its familiar disguise: *"A full SPA framework adopted for an admin console that is forms, tables, and four workflows"*. A framework or library dependency must provide durable leverage greater than its coupling cost, lifecycle risk (major-version migrations on the framework's schedule, not yours), build complexity, debugging cost, and portability loss. Convenience alone is not enough when platform-native contracts are sufficient — and for UI units, Web

Components + small libraries cover a large share of operator-console needs with zero framework lock. This is not “never React”: it is prove the dependency earns its rent, in writing, before it owns your contract surface.

The discipline. Adopting a framework/major library requires a stated case: what it provides that platform contracts cannot, what it costs across coupling/lifecycle/build/debug/portability, and what the exit path is. Re-justify at major-version bumps — the rent is recurring, not one-time.

See also: 78, 115

121. A value whose meaning is bound to identity, hidden mutable state, or single ownership must not be copied, aliased, or shared without an explicit ownership/fencing rule

CRITICAL SEVERITY.

The trap. A struct/object/regex/buffer looks like plain data, so copying or sharing it is free — but its meaning lives in its identity and mutable internals, not its visible shape. Its familiar disguise: *“It’s just a value — copy the struct, reuse the instance, share the object across handlers”*. Some values carry meaning in their IDENTITY and hidden mutable state, not in their visible fields: a mutex or sync primitive (lock state is bound to the object’s address), a stateful cursor (a global-flag regex’s lastIndex, an iterator), a buffer/slice backing array, a message handed to another consumer, a config object owned by one environment. Copying such a value duplicates the SHAPE while silently forking or aliasing the hidden STATE: two “locks” that no longer exclude, two readers mutating one shared message, one cursor advanced by concurrent callers, two environments sharing one mutable config. The result is a data race, lost mutual exclusion, corrupted iteration, or cross-owner mutation — bugs that pass local tests and fail under concurrency or scale.

The discipline. Before copying, reusing, or sharing a value, ask: does its meaning depend on identity, hidden mutable state, synchronization, cursor position, or single-owner lifecycle? If so, the copy/share needs an EXPLICIT ownership or fencing rule — pass a pointer/reference to the single owner; give each concurrent caller its own instance; defensively copy before hand-off; fence with a lock the copy cannot bypass. Lean on static help where it exists (Go’s copylock vet check; lint rules against shared stateful regexes).

See also: 3, 4, 25

122. Every projection must be rebuildable from owner truth — if you can’t delete and regenerate it, it has secretly become state

HIGH SEVERITY.

The trap. A generated artifact that holds the only copy of a fact feels like a source — it is a leak. Its familiar disguise: *“This generated file drifted, so I’ll hand-edit it to match reality”*. Envoy config, DNS records, /etc/hosts entries, service config files, package runtime files, and dashboards are disposable projections of owner state, not authority. If a projection cannot be deleted and regenerated from the owning actor’s truth, the projection has secretly become state and authority is leaking into a build output.

The discipline. Falsifiable test: delete the projection, run reconcile. If the system cannot restore it from owner truth, authority has leaked and must be moved back to the owner. Projections must never be hand-edited as the fix; the fix is to correct owner state and regenerate.

See also: 1, 127

> *Field note — Every wrapper that hides nothing is a small tax, collected forever, on everyone who reads the code after you.*

VIII. On Change, Governance, and the Releasable Road

The main branch must remain forever releasable, for a thousand small conveniences become one large ruin the day they are all due at once. Change the intent before the structure; let discovery propose and only a human promote. And know the last law, above all the green tests: there is no resolution without a respected contract.

123. Main branch must remain releasable — every merge preserves build, tests, graph validity, and artifact freshness

HIGH SEVERITY.

The trap. A red trunk feels like a temporary, private inconvenience — "it's just my WIP, I'll fix CI in the next commit." But a non-releasable main branch is a shared liability — every other change now builds on an unknown-good base, bisection is poisoned, and "is this change safe?" becomes unanswerable for everyone, not just you. The releasable invariant is precisely what makes small, frequent, low-fear merges possible at all. Its familiar disguise: *"Merge it now, green it up later — the branch is broken but only for a little while"*. Continuous Delivery's root law: the mainline is kept in a state where it COULD be released at any commit. Not "released continuously" — RELEASABLE continuously. Each merge must preserve the properties that make a release safe, so the distance from "merged" to "shippable" stays near zero.

The discipline. The releasable properties are enforced as a merge gate, not a post-merge cleanup task. A change may merge only when build, tests, graph validation, and artifact freshness all pass on the merge result.

See also: 6, 41, 126

124. A large or risky change must decompose into reviewable, behavior-preserving slices

WARNING SEVERITY.

The trap. Splitting a finished change feels like make-work — the code already exists, why carve it up? But review quality collapses non-linearly with diff size — past a few hundred lines reviewers skim, and "is this change reviewable?" silently becomes "no." Small slices are not bureaucracy; they are how a human can actually certify that a change is safe. Its familiar disguise: *"One giant PR that touches twelve subsystems because the feature 'is all one thing'"*. Trunk-based development's working assumption: changes arrive in small, independently reviewable, behavior-preserving increments. The point is not smallness for its own sake — it is that each slice can be understood, reviewed, tested, and reverted on its own, keeping the trunk releasable between slices (see main_branch_must_remain_releasable).

The discipline. Decomposition is a proposal, not a hard gate: AWG may surface "this change is large and crosses N boundaries / M critical invariants — consider slicing" as a reviewable candidate. It must not silently block or auto-split. Some changes are irreducibly atomic (a single rename across all call sites); when a large diff is genuinely indivisible, that is the declared reason, owned and bounded (exception_must_have_reason_owner_and_expiry).

See also: 52, 123

125. A high-risk change must carry test evidence — or a documented, owned, expiring exception

HIGH SEVERITY.

The trap. For risky code, manual confidence feels sufficient because the change "looks obviously correct" — but the risk of a change is not the size of its diff, it is the severity of what it can break. A one-line change to an authority, install/update/join, persistence, or control-plane path can take down trust or availability, and "I checked it by hand" is not evidence anyone else can re-run. Its familiar disguise: *"It's a small change to the auth path, I tested it manually, ship it"*. Continuous Delivery treats automated tests as the evidence that lets a change move without ceremony. The corollary: where the blast radius is large, the evidence requirement is non-negotiable. A change is high-risk when it touches critical/high-severity invariants, authority/authorization paths, install/update/join lifecycle, security, persistence, or control-plane behavior.

The discipline. High-risk changes merge with test evidence attached, or with an explicit exception carrying reason, owner, and expiry/review condition (exception_must_have_reason_owner_and_expiry) — never with silence.

See also: 6, 9, 35

126. Generated artifacts must be fresh against their source before merge

HIGH SEVERITY.

The trap. A regenerated artifact feels like a derived afterthought you can refresh whenever — but a stale generated file is a lie about its source — the graph, the stubs, or the docs now disagree with the code they claim to describe. Anyone (human or agent) who reads the artifact instead of re-deriving it gets a false picture, and the divergence compounds silently until a rebuild surprises everyone. Its familiar disguise: *"The .nt seed / proto stubs / generated YAML are a bit stale, but the source change is what matters"*. Deterministic, reproducible builds require that every artifact derived from source matches that source at merge time. For an AWG project the derived artifacts include the graph seed (awareness.nt), proto outputs, inferred contract/import YAML, and generated docs — each produced from a source by a deterministic generator.

The discipline. Every generated artifact ships with a deterministic regenerator and a –check (or freshness-gate) mode wired into CI; the merge gate fails on owned staleness. Generated files are produced ONLY by their generator and committed from it — never hand-edited (the file headers say so for a reason).

See also: 5, 49, 123

127. Graph truth enters only through approved corpus artifacts and a deterministic rebuild — never a live side channel

CRITICAL SEVERITY.

The trap. A live write feels faster and harmless — the fact is correct, why route it through a YAML file and a rebuild? Because a fact's TRUST comes from its path, not its content — a triple inserted out-of-band has no review, no provenance, no source file, and cannot be reproduced by a rebuild. The moment the store can diverge from the corpus, "is this graph trustworthy?" can no longer be answered yes. Its familiar disguise: *"Just insert the triple into the running store so the agent sees it now"*. The trust model is a one-way pipeline: human intent → principle → invariant → evidence → candidate/proposal → human approval → corpus file → deterministic rebuild → graph truth Every trusted fact in the graph is reproducible by rebuilding from the

committed corpus. That reproducibility IS the trust: anyone can regenerate the seed and get the same graph, and every node traces to an authored, reviewed source file.

The discipline. There is no live graph-write path for trusted facts. Discovery produces candidates (discovery_produces_candidates_not_facts); promotion is the ONLY way in, and promotion = append-to-corpus + validate + deterministic rebuild. Runtime additive loads (e.g. previewing a foreign repo) must never touch the committed seed and must be clearly non-canonical.

See also: 1, 40, 128

128. Discovery produces candidates, not facts — automated extraction may propose, only humans promote

CRITICAL SEVERITY.

The trap. High extractor confidence feels like truth — if the scan is usually right, why make a human approve each one? Because confidence is not authority. An audit, source scan, intent miner, or cold-source extractor sees correlation and plausibility, not intent; promoting its output directly makes the graph an unreviewed guess wearing the costume of fact, and the first wrong-but-confident candidate poisons everything downstream that trusted it. Its familiar disguise: *"The miner found 12 intents with high confidence, auto-land them in the graph"*. The human-approval gate is the load-bearing wall of the trust model. Every automated producer of knowledge — audit, source-check, intent mining, cold-source extraction, pattern inference — emits CANDIDATES: proposals marked status:candidate, carrying confidence and provenance, written to a candidates area, awaiting review. None of them may create trusted graph truth on their own.

The discipline. Discovery tools emit candidates only: status:candidate, with confidence and discovered_from provenance, in the candidates area. Promotion is a separate, explicit, human-initiated step that validates and appends to the corpus, then rebuilds. Confidence may RANK candidates; it may never auto-promote them.

See also: 21, 22, 127

129. A change to runtime behavior must ship an observable path — if it can happen, an operator must be able to see that it happened

HIGH SEVERITY.

The trap. Observability feels like a follow-up you bolt on after the behavior works — but a runtime change with no observable result is a change you cannot operate or debug. When it misbehaves in production there is nothing to look at, and "is this change observable?" is answered "no" exactly when you most need it to be yes. The signal must be born with the behavior, not retrofitted after the incident. Its familiar disguise: *"Added the new lifecycle transition; we'll add logging/metrics if someone reports a problem"*. DevOps fuses change with operability: you build it, you run it, so a change to runtime behavior must expose how to observe its effect. Any change to lifecycle, installation, update, join, service health, or failure handling must surface an observable result — a log at appropriate severity, a metric, a state transition, or an operator-visible status — plus a test that asserts the observable appears.

The discipline. Runtime-behavior changes ship with their observable path and a test that asserts it. The observable names what happened in terms an operator can act on, and its timeliness matches the decision it informs.

See also: 31, 32, 41

130. Cross-repo and cross-artifact references must be validated before merge — a cited anchor must resolve to a defined one

HIGH SEVERITY.

The trap. A reference across a repo or artifact boundary feels safe because it looked right when written — but the target moves, renames, or never existed, and an unvalidated reference is a dangling pointer in the knowledge graph. It reads as a real link, so consumers trust a connection that resolves to nothing, and the rot stays invisible until something follows the broken edge. Its familiar disguise: *"It references a symbol/invariant/contract in the other repo; it'll line up eventually"*. Integration safety: every reference that crosses a boundary — repo, file, or artifact — must be checked to resolve before it is trusted. Cross-repo links, source anchors, contract references, and related-* edges must each point at a definition that actually exists; a cite without a matching define is a dangling reference.

The discipline. A merge gate validates references and fails on newly introduced dangling references or missing source files. Known, temporary cross-repo gaps live in an explicit allowed-dangling baseline naming the drift they cover; the gate fails only on drift beyond it.

See also: 5, 23, 126

131. Every exception, suppression, or allowlisted violation carries a reason, an owner, and an expiry or review condition

HIGH SEVERITY.

The trap. A suppression feels like a tidy way to get unblocked now — the violation is known, you'll deal with it later. But an unannotated exception is debt with no creditor — no one knows why it exists, who owns paying it down, or when it should be gone, so it becomes permanent. Allowlists that only ever grow are how a system's invariants quietly stop meaning anything. Its familiar disguise: *"Add it to the allowlist / // nolint / skip the test — quietly, with no note"*. Lean debt control: incurring debt can be rational, but UNTRACKED debt is not. Every deliberate deviation — an allowlist entry, a bypass, a temporary suppression, a skipped/xfail test, a known-violation baseline entry, a TODO that defers a real fix — must record three things: the REASON it exists, the OWNER accountable for it, and an EXPIRY or review condition saying when it must be reconsidered.

The discipline. Allowlists, baselines, suppressions, and deferral markers require structured metadata: reason, owner, and an expiry date or review condition. Entries missing any of the three are themselves findings. Tooling that consumes an allowlist should be able to report entries that are expired or unowned so the list can shrink.

See also: 7, 9, 45

132. Architectural intent must change before the structure does — update the decision, then let the graph accept the new shape

HIGH SEVERITY.

The trap. Code-first feels honest — ship the real change, document the reasoning later. But when structure that alters authority, ownership, boundaries, lifecycle, or contract semantics moves ahead of its stated intent, the graph's "healthy" picture is now describing an architecture nobody decided on. The drift looks sanctioned because the structure exists; intent written afterward is rationalization, not a decision, and the why is lost for

everyone who comes after. Its familiar disguise: *"Refactor the boundary now; we'll update the ADR / intent docs afterwards if we remember"*. The ADR discipline: a structural change to the architecture is preceded by a recorded decision about its intent. When a change alters authority, ownership, boundaries, lifecycle, or contract semantics, the intent — decision record, awareness annotations, or an ADR-like corpus entry, plus the affected invariants — is updated FIRST (or with the change), so the graph accepts the new shape as healthy only once a human has stated why it is the intended shape.

The discipline. Structural changes to authority/ownership/boundaries/lifecycle/contract semantics land together with the updated intent that sanctions them — the decision/annotation/invariant change is part of the same reviewable unit, not a deferred follow-up. The graph treats a new structural shape as healthy only once its intent is recorded.

See also: 1, 47, 80

133. Each instance pins its state-machine version — definition changes never silently reinterpret running instances

HIGH SEVERITY.

The trap. Shipping the new state graph feels like an upgrade — it orphans in-flight instances whose states no longer exist. Its familiar disguise: *"new code removes WAITING_VERIFY while old workflows are sitting in it"*. A workflow has a definition (states, transitions, guards, retry policies, activity handlers). Changing it must not silently reinterpret old instances. Pin workflow_definition_version per instance; incompatible state-graph changes require an explicit migration.

The discipline. Each running instance records the definition version it started under. Incompatible definition changes are gated by a migration, not applied in place.

See also: 47, 54

> *Field note — 'We'll mechanize it later' is the most expensive sentence in engineering, because 'later' is denominated in incidents.*

Afterword — The Architecture That Remembers

You have now read one hundred and thirty-three ways to be wrong with total confidence. Take heart: they are not one hundred and thirty-three unrelated mistakes. They are a small number of *shapes*, recurring — a fallback that hides a failure by wearing truth's face; a write with no appointed keeper; two writers racing on one field; an intermediate state that satisfies a completeness check; a green light standing in for a proof no one performed. Learn to see the shape and you can find the next instance before it fires, in code that has not yet broken.

But here is the harder truth the old strategists knew and we keep forgetting: knowledge that lives only in a person leaves when the person does. The maxims in this book were, for most of software's history, unwritten — carried in the heads of the three engineers who were on the incident call, in a post-mortem nobody re-reads, in a review comment that scrolled out of history. The next contributor could not see them, and so made the reasonable-looking change that quietly broke one, and the system drifted a little further from its own design.

That is no longer only a human problem. An AI agent arrives at your repository every session with a flawless reading of the syntax and no memory of the architecture. It will write a patch that compiles, passes the tests it can see, reads beautifully — and violates a law that was in none of the files it opened. A stronger model reads the code better; it still cannot read what the code does not contain.

So the final maxim is not in the list, because it is about the list itself: **write the memory down where the work happens, and make it answer at the moment of the edit.** That is what the Awareness Graph does with these principles — it compiles them into a graph the repository carries, and serves the ones that apply to the file you are about to change, before you change it, to human and agent alike. The counsel arrives before the mistake, not after.

The highest skill was never to write the most code the fastest. It is to keep the system knowing what it is — while everyone who once knew is asleep.

Index of the 133 Principles

The maxims above are paraphrase and field-craft. Below are the principles themselves, by their true identifiers, as they live — machine-queryable, with their enforcement tiers and the scars of the incidents that taught them — in `cmd/awg/templates/awareness/meta_principles.yaml`.

I. On Authority and the Ownership of Truth

1. `meta.storage_is_not_semantic_authority` — Storage is not semantic authority — truth belongs to the owning actor, not the backing store
2. `meta.identity_computation_must_be_invariant` — Identity computation must be invariant — one field, one meaning, one canonical computation, everywhere
3. `meta.competing_writers_must_converge_or_be_fenced` — Distributed actors doing the same job must converge, not compete — two writers with different state will fight until one wins by accident
4. `meta.structure_must_not_be_stripped_in_projection` — Meaning lives in structure — a value projected down to its primitive shape carries the lie of universality
5. `meta.code_must_not_mirror_external_enumerations` — Code mirrors of external truth drift silently — derive the set, don't author it
6. `meta.end_to_end_check_is_the_only_truth` — The only authoritative correctness check is at the actor that owns the cross-layer intent
7. `meta.bounded_staleness_must_be_named_not_assumed` — Cached, replicated, or derived state must declare its staleness contract — 'eventually consistent' is a category, not a contract
8. `meta.physical_clocks_disagree_use_logical_ordering` — Physical clocks on different nodes never agree precisely — for causal ordering, use logical clocks; for total order, use consensus
9. `meta.fail_safe_defaults_when_authority_is_uncertain` — When the authority decision cannot be made, the default is REFUSE — never 'allow because we could not check'
10. `meta.least_privilege_is_not_a_default_it_is_an_explicit_grant` — Every actor's privileges must be explicit and minimal — the default for a new actor is NO privileges, not 'whatever was convenient at creation time'
11. `meta.power_hiding_hurts_at_the_edges` — Abstractions that hide engine choices from their callers force callers to either tolerate the hidden choice or bypass the abstraction entirely
12. `meta.uniform_naming_reduces_conceptual_load` — All system entities should be named by a uniform scheme — heterogeneous naming creates friction at every API boundary
13. `meta.capability_is_not_intent` — Capability is not intent — an installed binary answers 'can this node run X', never 'should it'
14. `meta.membership_is_admitted_not_self_declared` — Membership is admitted, not self-declared — a service or discovered peer never decides its own place in topology
15. `meta.reconciler_repairs_only_within_its_authority_lane` — Each actor repairs only inside its authority lane — observation flows up, decision flows down, projection renders outward
16. `meta.workflow.command_is_not_state` — Commands request transitions — they do not become state
17. `meta.workflow.instance_has_single_transition_writer` — A workflow instance has a single transition writer — only one authority advances it
18. `meta.distributed_data_topology_is_correctness` — A distributed data substrate's topology is part of its correctness, not a runtime detail — running is not the same as safe

19. `meta.ha_requires_failure_tolerance_not_member_count` — High availability is defined by tolerated failure, not by how many members are present
20. `meta.limited_members_are_not_capacity` — Non-voting, non-owning, learner, observer, limited, or partially-initialized members do not count as capacity

II. On Signals, Silence, and Honest Uncertainty

21. `meta.fallback_must_degrade_semantics` — Fallback must degrade semantics — a fallback that returns the same shape as truth will be mistaken for truth
22. `meta.authority_must_express_uncertainty` — Authority must express uncertainty — if the owner cannot say ‘unknown’, callers will turn silence into lies
23. `meta.absence_scope_must_be_explicit` — Absence scope must be explicit — ‘not found where’ is not the same as ‘does not exist’
24. `meta.connection_errors_must_not_be_absorbed` — Errors must not be silent on connection paths — a connection error absorbed into a timeout is an invisible outage
25. `meta.assertions_must_carry_their_scope` — Every assertion — positive or negative — carries the scope of its truth; aggregation without naming the scope is a lie
26. `meta.abstraction_must_not_hide_unmeasured_cost` — An abstraction that swallows its own failure mode is worse than no abstraction
27. `meta.timeout_is_a_decision_not_a_truth` — A timeout means ‘I did not hear back in time’ — never ‘the other side failed’
28. `meta.timestamp_is_an_observation_not_an_event_time` — A recorded timestamp is the observer’s clock at moment of observation — not when the event actually happened
29. `meta.clock_skew_invalidates_cross_node_time_comparison` — Comparing timestamps from two nodes without a skew bound is structurally unsound — Spanner’s TrueTime made this explicit by returning uncertainty intervals, not points
30. `meta.principal_must_survive_proxy_chains` — When auth flows through a proxy, the downstream service must know WHO originated the request — not WHO proxied it
31. `meta.observability_must_match_decision_horizon` — The granularity and freshness of emitted signal must match the granularity and freshness of the decisions that depend on it
32. `meta.alert_must_name_a_violated_invariant_not_a_crossed_threshold` — Alerts that fire on numerical thresholds produce noise during normal spikes and silence during creative failures — alerts should name failed invariants
33. `meta.metric_aggregation_destroys_actionable_signal` — Aggregation summarizes; aggregation also destroys — the individual events that contained the actionable signal are gone
34. `meta.harvest_and_yield_are_distinct_availability_dimensions` — Yield (queries completed) and harvest (data per answer) are two separate axes of availability — preserving one says nothing about the other
35. `meta.negative_result_requires_coverage_attestation` — ‘Nothing found’ is not a finding — the observer must distinguish ‘searched and verified empty’ from ‘never searched’
36. `meta.event_is_notification_not_authority` — An event is a notification to go ask the owner — never the authoritative state itself
37. `meta.workflow.failure_state_is_classified` — Failure is classified into actionable states, not collapsed to FAILED
38. `meta.workflow.observation_is_not_transition_authority` — Observations are evidence, not transitions — a probe never directly mutates state
39. `meta.control_plane_and_data_plane_availability_are_distinct` — Control-plane availability, data-plane availability, durability, and convergence are distinct dimensions and must be reported separately

III. On Time, State, and the Long March of Work

40. `meta.write_creates_completion_obligation` — Every write is a promise — an unfinished promise is a blockage
41. `meta.half_done_must_not_look_done` — Half-done must never look done — intermediate state must not satisfy completeness predicates
42. `meta.silence_is_not_valid_for_unexpected` — Silence is not a valid response to the unexpected — unhandled cases must fail closed
43. `meta.failure_response_must_contract_not_amplify` — Failure response must contract, not amplify — retry, re-enqueue, and re-emit must be bounded, or the response becomes the outage
44. `meta.diagnostic_output_must_be_bounded` — Diagnostic output must be bounded — one error must not become N records, or the diagnosis becomes a harder failure than the disease
45. `meta.binding_outlives_evidence_until_invalidated` — Every binding carries 'I checked this then' — when 'now' is different from 'then', the binding is a phantom unless re-validated
46. `meta.state_mutations_must_be_durably_committed_before_side_effects` — Intent commits before action, retry is the only response to failure — no alternative paths once committed
47. `meta.early_consolidation_ossifies_uncertainty` — Code that crystallizes before its true shape is understood becomes cement around its bug
48. `meta.bad_path_must_make_progress` — Failure response must move the system toward a terminal state, not freeze it indefinitely
49. `meta.idempotence_is_a_requirement_not_a_quality` — Every operation that can be retried, replayed, or re-dispatched MUST be idempotent — there is no 'might be retried'
50. `meta.duration_versus_deadline_is_not_interchangeable` — 'Wait 30 seconds' and 'wait until 13:42:30' mean different things across restarts, pauses, and NTP corrections — the choice is semantic
51. `meta.authorization_check_is_a_snapshot_not_a_promise` — An authorization decision is bounded by the moment it was made — the underlying authority can change before the action is taken
52. `meta.feedback_loop_must_be_faster_than_drift` — A control loop running slower than the drift it is meant to correct will oscillate or diverge — never converge
53. `meta.checkpoint_alone_is_inadequate_for_growing_state` — Atomic single-write of state is necessary but not sufficient — growing or large state needs append-only logs alongside the checkpoint
54. `meta.adoption_requires_running_existing_systems_unchanged` — Users will not adopt a new system that requires complete replacement of their existing one — adoption demands migration paths, not capability superiority
55. `meta.MTTR_focus_outperforms_MTBF_for_evolving_systems` — For systems that change frequently, repair time is much easier to improve than failure rate — and has equal impact on uptime
56. `meta.graceful_degradation_is_the_normal_mode_not_an_exception` — Saturation is the normal operating mode for production systems — graceful degradation is the explicit response, not an emergency fallback
57. `meta.cleanup_must_be_owned_and_reversible` — Every install action has a matching uninstall owner — cleanup removes only what a package/profile/generation owns
58. `meta.recovery_mode_must_be_explicit_and_narrower` — Recovery mode is explicit, evidence-gated, and strictly narrower than normal mode — emergency shortcuts must never become normal paths
59. `meta.workflow.state_is_declared_machine_state` — Workflow state is a declared machine state, not status text
60. `meta.workflow.transitions_are_explicit` — Every transition is explicit — no hidden state jump
61. `meta.workflow.transition_is_atomic_with_history` — State transition and its evidence event must be

one atomic durable write

62. `meta.workflow.history_is_append_only` — Workflow history is append-only — you append corrections, never rewrite the past

63. `meta.workflow.current_state_is_rebuildable_projection` — Current state is a rebuildable projection of the history log

64. `meta.workflow.transition_guards_are_named_contracts` — Transition guards are named contracts, not scattered if-statements

65. `meta.workflow.boundary_commands_are_idempotent` — Every command/callback at a boundary is idempotent — any external signal can arrive twice

66. `meta.workflow.commands_are_generation_guarded` — Mutating commands carry the generation they observed — stale actors cannot move state

67. `meta.workflow.terminal_state_is_final_without_recovery_contract` — Terminal states are final except through an explicit recovery contract

68. `meta.workflow.side_effects_cross_outbox_boundary` — External side effects cross an outbox/activity boundary, never interleave with state

69. `meta.workflow.time_is_durable_event` — Time is a durable event, not a sleep — timers are persisted, not implied by wall-clock waits

70. `meta.workflow.compensation_is_forward_recovery` — Compensation is forward motion — rollback is fantasy once a side effect escaped

71. `meta.workflow.retry_is_modeled_state` — Retry is modeled state — attempt, delay, reason, and idempotency, not a hidden code loop

72. `meta.workflow.unknown_must_not_be_invented` — Unknown is safer than invented — a workflow that can't prove truth enters UNKNOWN, it does not guess

73. `meta.workflow.projection_must_not_drive_progression` — Projections must never drive progression — UI/queue/CLI/log views observe, they do not advance the workflow

74. `meta.quorum_change_requires_pre_failure_safety` — A quorum/ownership-changing operation must be safe if the joining or leaving member fails halfway through

75. `meta.membership_promotion_requires_readiness_proof` — A member must not be promoted into a topology-critical role until it has proven substrate-specific readiness

76. `meta.recovery_must_be_substrate_specific` — Distributed-data recovery must use the substrate's own recovery model, not a generic restart/delete/reinstall

77. `meta.topology_policy_must_be_explicit_before_automation` — Before automating membership changes for a data substrate, its allowed/forbidden/transitional topology states must be defined — otherwise refuse destructive or quorum-changing actions

IV. On Dependencies, Topology, and Retreat

78. `meta.critical_path_no_non_critical_dependency` — The critical path must not depend on non-critical services — a dependency you don't need will kill you on the recovery path

79. `meta.circular_dependency_must_have_break_glass` — Every circular dependency must have a break-glass path that doesn't go through the cycle — a system that deploys itself must have a path that doesn't go through itself

80. `meta.topology_change_is_a_first_class_event` — Cluster topology is a dynamic input — code that caches it at startup operates on a snapshot that becomes a lie within minutes

81. `meta.partition_response_must_be_predeclared` — An actor's response to network partition must be decided BEFORE the partition occurs — choosing in the moment is the bug

82. `meta.mobility_is_stronger_recovery_than_replication` — State that can move between nodes

recovers via rebind; state that only replicates recovers via reinstall — rebind is much cheaper

83. `meta.replication_vs_partitioning_chooses_availability_dimension` — Replication preserves harvest under fault and reduces yield; partitioning preserves yield and reduces harvest — the choice maps to which dimension the workload tolerates losing

84. `meta.load_redirection_must_be_explicit_capacity_planning` — Replication does not preserve yield unless surviving nodes have spare capacity to absorb redirected load — and that capacity must be measured, not assumed

V. On the Operator's Eye

85. `meta.ui.screen_claim_must_bind_to_authority` — Every screen claim binds to the correct authority — desired, cached, optimistic, and confirmed state must not collapse into one visual meaning

86. `meta.ui.state_certainty_must_be_visually_distinct` — Certainty is part of the value — loading, stale, unknown, optimistic, and confirmed must each look like what they are

87. `meta.ui.same_truth_same_language` — One meaning, one visual language — the same operational state must render identically everywhere it appears

88. `meta.ui.destructive_action_requires_confirmed_authority` — An enabled destructive button is an assertion — permission and risk state must be confirmed before the action is offered

89. `meta.ui.failure_must_preserve_diagnostic_context` — Failure is when the screen matters most — errors must preserve context and offer a path, never blank the operator's world

90. `meta.ui.provenance_over_recall` — Operational claims carry their provenance — source, node, generation, and version are shown, not remembered

91. `meta.ui.meaning_must_survive_presentation_loss` — Meaning must survive presentation loss — color removal, layout collapse, keyboard-only, and theme changes must not change what the screen asserts

92. `meta.ui.decoration_must_not_impersonate_authority` — Decoration must not impersonate authority — cosmetic elements and generated prose stay visually subordinate to authoritative state

93. `meta.ui.simplicity_must_not_hide_operational_truth` — Simplicity must not hide operational truth — a clean lie is worse than a messy truth

94. `meta.ui.operator_must_remain_in_control` — The operator remains in control — automation, auto-refresh, optimistic updates, and AI assistance must not change state without visible intent and consent

95. `meta.ui.control_must_be_reversible_or_guarded` — Dangerous actions are reversible or guarded — undo, dry-run, preflight, or explicit risk display before execution, with no fast path around the guard

96. `meta.ui.workflow_must_yield_closure` — Every workflow yields closure — the final state, receipt, or required next action must be visible; a disappeared spinner is not a result

97. `meta.ui.ai_assistance_must_be_explainable_and_bounded` — AI assistance is explainable and bounded — summaries cite source, time, and confidence; recommendations show the evidence boundary

98. `meta.ui.task_path_must_match_operator_goal` — The screen serves the operator's task, not the system's schema — API shape is not information architecture

99. `meta.ui.notification_volume_must_match_operator_capacity` — Notification volume must match operator capacity — an alarm flood is a perception outage

100. `meta.ui.interactive_element_must_have_stable_identity` — Elements that carry operational meaning must be individually addressable — a styled span with no identity is invisible to tooling, tests, and the operator's tools

101. `meta.contract_must_be_explicit_before_resolution` — The governing contract must be made explicit before a change can be called resolved

102. `meta.contract_must_define_repair_scope` — A repair contract must define not only the intended rule, but also the scope in which the rule may be applied

103. `meta.no_resolution_without_a_respected_contract` — No resolution without a respected contract — green tests are not enough

VI. On Arrangement, Weight, and Visual Command

104. `meta.ui.visual_hierarchy_must_match_decision_hierarchy` — Visual weight, order, spacing, and grouping must match the operator's decision path — safety evidence outranks decoration, always
105. `meta.ui.visual_grouping_must_match_semantic_grouping` — Visually grouped elements must share real semantic relationship — layout must not tell a story the data does not
106. `meta.ui.spacing_must_encode_relationships` — Spacing is information — equal spacing makes unrelated facts look related, and grouping gaps must mean grouping
107. `meta.ui.proportion_must_reflect_operational_weight` — Screen area and interaction prominence must reflect operational importance — critical evidence is never smaller than decoration
108. `meta.ui.color_must_have_semantic_contract` — Status colors have stable contracts — success means confirmed runtime truth only, and no color serves two conflicting roles
109. `meta.ui.typography_must_express_information_hierarchy` — Type expresses the information hierarchy — title, status, evidence, warning, and metadata must be visually distinguishable at a glance
110. `meta.ui.theme_tokens_must_encode_roles_not_preferences` — Theme tokens encode semantic roles, not preferences — components consume success/warning/danger/stale/unknown, never raw values

VII. On Boundaries, Reuse, and the Shape of Code

111. `meta.code.reusable_unit_must_have_a_stable_semantic_boundary` — A reusable unit is a stable semantic concept — explicit contract, hidden complexity, owned lifecycle, inspectable behavior, explicit extension
112. `meta.code.contract_must_outlive_implementation_fashion` — Contracts outlive implementation fashion — callers depend on semantic inputs/outputs/events, never private structure
113. `meta.code.complexity_must_be_hidden_behind_honest_boundaries` — Hide complexity, never truth — the boundary must expose the states, failures, side effects, and authority assumptions callers need
114. `meta.code.reuse_must_follow_semantic_cohesion` — Reuse follows meaning, not resemblance — shared code represents one coherent concept, never a visual or structural coincidence
115. `meta.code.composition_must_prefer_standard_protocols` — Compose through standard protocols first — platform contracts (DOM, events, HTTP, gRPC, a11y) before private framework mechanisms
116. `meta.code.debuggability_is_part_of_correctness` — Debuggability is correctness — structure must preserve inspection, tracing, and a path from runtime behavior back to source intent
117. `meta.code.local_state_must_not_become_hidden_authority` — Local state caches and stages — it never becomes the authority for domain truth, permission, completion, or health
118. `meta.code.extension_points_must_be_explicit` — Variation happens at declared extension points — slots, properties, events, adapters — never by reaching into private internals
119. `meta.code.abstraction_must_be_deeper_than_its_interface` — Modules must be deep — an abstraction earns existence by hiding more complexity than its interface adds
120. `meta.code.framework_dependency_must_be_earned` — A framework dependency must earn its rent — durable leverage greater than its coupling, lifecycle, build, debugging, and portability costs
121. `meta.code.identity_bound_state_must_not_be_copied` — A value whose meaning is bound to identity, hidden mutable state, or single ownership must not be copied, aliased, or shared without an explicit ownership/fencing rule
122. `meta.projection_must_be_rebuildable_from_owner` — Every projection must be rebuildable from owner truth — if you can't delete and regenerate it, it has secretly become state

VIII. On Change, Governance, and the Releasable Road

123. `meta.main_branch_must_remain_releasable` — Main branch must remain releasable — every merge preserves build, tests, graph validity, and artifact freshness
124. `meta.change_must_be_split_into_reviewable_slices` — A large or risky change must decompose into reviewable, behavior-preserving slices
125. `meta.high_risk_change_requires_test_evidence` — A high-risk change must carry test evidence — or a documented, owned, expiring exception
126. `meta.generated_artifacts_must_be_fresh_before_merge` — Generated artifacts must be fresh against their source before merge
127. `meta.graph_truth_must_come_from_approved_corpus` — Graph truth enters only through approved corpus artifacts and a deterministic rebuild — never a live side channel
128. `meta.discovery_produces_candidates_not_facts` — Discovery produces candidates, not facts — automated extraction may propose, only humans promote
129. `meta.runtime_change_requires_observability_path` — A change to runtime behavior must ship an observable path — if it can happen, an operator must be able to see that it happened
130. `meta.cross_repo_reference_must_be_validated` — Cross-repo and cross-artifact references must be validated before merge — a cited anchor must resolve to a defined one
131. `meta.exception_must_have_reason_owner_and_expiry` — Every exception, suppression, or allowlisted violation carries a reason, an owner, and an expiry or review condition
132. `meta.architectural_intent_must_change_before_structural_drift` — Architectural intent must change before the structure does — update the decision, then let the graph accept the new shape
133. `meta.workflow.state_machine_version_is_pinned` — Each instance pins its state-machine version — definition changes never silently reinterpret running instances

---

The Art of Software Architecture is drawn from the Awareness Graph (AWG), open source at github.com/globularios/awareness-graph. The 133 meta-principles were distilled from real production incidents on the Globular platform and are shipped as portable, domain-independent seed knowledge with every `awg init`*. What bit us is provenance; what we learned belongs to everyone.*